Online Decision Trees for Varying Feature Spaces via Gradient-Based Surrogates

Kaibu Feng* Heitor Murilo Gomes† Xingquan Zhu‡ Yi He**

Abstract. This paper proposes DynaGradTree, a novel online decision tree model that can handle data streams with Varying Feature Space (VFS). The key insight of our design is that soft feature selection naturally enables learned surrogate splits, such that when a sample lacks the optimal split feature, the model automatically selects the best available alternative from a learned preference ranking. Unlike static, post-hoc surrogates, these preferences emerge from end-to-end gradient optimization. DynaGradTree leverages dynamic parameter initialization for feature space growth and instance-level masking for sample-specific feature availability, and achieves first or second place on 19 of 28 comparisons against traditional online trees and VFS-specific online learning methods across 14 real-world datasets. Ablation studies and surrogate feature testing further validate that such performance improvements stem from interpretable learned substitution strategies. Our code is available at github.com/fengkaibu1/SIAM_DynaGradTree.

1 Introduction Decision trees and tree-based ensembles remain among the most widely used interpretable machine learning models (10; 20), with critical applications spanning clinical decision support (44), financial risk modeling (17), and network security (2), where model transparency is essential for regulatory compliance and human oversight. Modern applications increasingly require models to operate in online scenarios, where data arrives continuously as streams (11; 14) and models must adapt in real-time without costly retraining. This shift from batch to online learning introduces new challenges, especially when the available features themselves change over time.

A particularly challenging scenario arises in Varying Feature Spaces (VFS), also known as open feature spaces (22), where different samples may access different feature sets and the feature space grows as new features emerge. Traditional decision tree methods face fundamental limitations in such settings. In batch learning, CART (8) introduced surrogate splits to handle unavailable features by maintaining backup features at each node, but this mechanism is computed post-hoc after tree construction and remains static once determined. For online learning, Hoeffding Tree family (11; 30; 6) and related methods (38) enable incremental tree construction from data streams, but they assume fixed feature spaces throughout the stream. When samples arrive with varying feature availability or when new features emerge over time, these methods require either data imputation or model reconstruction, both unsuitable for real-time streaming scenarios.

Recent work has addressed online learning with VFS through alternative approaches. Linear classifiers with feature projection (4; 53) adapt to changing features through weight updates, while ensemble methods (48; 46) distribute features across multiple models. Latent representation methods (21) learn shared embeddings to handle feature variations. While these methods successfully handle VFS scenarios, they sacrifice the interpretability that makes decision trees valuable for high-stakes applications. The tree structure provides transparent, auditable decision paths that practitioners can validate and explain, a property lost in aggregated ensemble predictions or learned latent spaces.

To address these limitations, this paper proposes DynaGradTree, a novel online decision tree method that handles VFS while preserving interpretable tree structure. Our key insight is that soft feature selection combined with instance-level masking naturally implements learned surrogate splits in continuous form. Unlike traditional tree methods that require fixed feature spaces, our gradient-based approach dynamically accommodates newly emerged features through parameter initialization and adapts to varying feature availability across samples through feature-level masking. Unlike VFS-specific methods that sacrifice interpretability for flexibility, our approach maintains transparent decision paths through hard, axis-aligned splits while jointly optimizing feature preferences across all tree parameters via end-to-end gradient descent.

Our contributions are: (1) We optimize the trade-

*Department of Data Science, William & Mary, Williamsburg, VA, USA ({kfeng03, yihe} @wm.edu).

†School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand (heitor.gomes@vuw.ac.nz).

‡Department of Electrical Engineering and Computer Science, Florida Atlantic University, Boca Raton, FL, USA (xzhu3@fau.edu).

*Corresponding Author. Email: yihe@wm.edu

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

off between VFS handling and interpretability in online tree models, enabling decision trees to accommodate dynamically evolving feature spaces while maintaining interpretable decision structures unlike existing approaches that sacrifice one for the other. (2) We introduce learned surrogate splits that replace static imputation or predetermined surrogates with dynamic, sample-adaptive feature selection, where tree nodes learn to rank and automatically select the most informative available feature for each sample based on its feature availability pattern. (3) We conduct extensive experiments demonstrating competitive performance against both online tree baselines and VFS-specific methods across 14 datasets, with ablation studies validating the learned surrogate mechanism.

2 Related Work

2.1 Online Learning with VFS Traditional online learning methods assume a fixed feature space in the data stream. Early work considered monotonically growing feature spaces, proposing algorithms that combine online learning with streaming feature selection (52; 53). Later studies further generalized this to arbitrarily VFS where features may emerge or vanish without predefined patterns (4; 23; 24). Feature evolution scenarios where old features fade and new features emerge were addressed through ensemble methods that recover vanished features (41; 42; 28). Recent work extended VFS learning to handle incomplete supervision (25), incomplete features with incomplete labels (51; 50), mixed-type data (21; 49), and combined challenges including concept drift and class imbalance (54; 35; 36). Existing methods for VFS learning can be categorized into two categories. Non-ensemble methods use linear classifiers with feature projection and adaptive weighting (4; 53), or use latent spaces representation, improving flexibility but lack interpretable decision structures (21; 49; 35; 36). Ensemble methods combine multiple models to handle feature variations, either through feature recovery (28), or forest-based aggregation (46; 48; 29). However, these ensemble methods aggregate predictions across multiple models, also resulting in low interpretability.

2.2 Online Decision Trees Online decision tree methods enable incremental learning from data streams. Hoeffding Trees (11; 27) pioneered this paradigm by using the Hoeffding bound to determine when sufficient statistics support confident split decisions, growing trees incrementally as data arrives. Following work extended this foundation to handle concept drift through concept-adapting mechanisms (30), adaptive tree replacement (6), eager splitting strategies (38), and ensemble diversity (7). Ensemble approaches further improved robustness through adaptive random forests (15), Mondrian processes (34), and ensemble voting schemes (5). More recent work has explored finer-grained structural operations including branch-level monitoring (1), subtree restructuring (26), global reorganization (37; 47), and random subspace sampling (16). Recent advances have integrated differentiable parameterizations (33) and gradient boosting for streams (18). However, these methods mostly assume fixed feature spaces, and while some can handle monotonically increasing features, no existing tree-based approach simultaneously addresses varying feature spaces where features can both emerge and disappear and handles feature availability patterns through mechanisms beyond simple imputation.

2.3 Differentiable Decision Trees Learning decision trees through gradient descent is a new method that came out in recent years. While soft decision trees (32; 13) and oblivious trees (45) enable backpropagation at the cost of interpretability or expressiveness, GradTree (40) preserves hard, axis-aligned splits through dense representation with differentiable parameters. However, GradTree was designed for batch learning with fixed feature spaces. Our work utilizes the gradient tree learning framework to handle VFS scenarios.

3 Preliminaries

3.1 Problem Formulation We consider online classification over data streams where the feature space varies across samples and evolves over time. The data stream $\mathcal{S} = \{(x_1, y_1), (x_2, y_2), \ldots\}$ consists of instance-label pairs arriving sequentially, where each label $y_t \in \{1, 2, \ldots, C\}$ belongs to one of C classes. Unlike traditional streaming settings that assume a fixed feature space, instances in VFS scenarios exhibit two key properties. First, different instances may observe different subsets of features. An instance x_t arriving at time t contains values for only a subset $F_t^{\text{obs}} \subseteq F_{\text{all}}$ of all possible features, where F_{all} denotes the complete feature space. Features in $F_{\text{all}} \setminus F_t^{\text{obs}}$ are unobserved for instance x_t. Second, the complete feature space F_{all} itself is not known in advance but grows as new features appear in the stream. At time t, the model has observed a cumulative feature set $F_t^{\text{cum}} = \bigcup_{i=1}^{t} F_i^{\text{obs}}$, where $|F_t^{\text{cum}}| = n_t$ denotes the number of distinct features encountered up to time t.

At each time step t, the decision tree produces a prediction $\hat{y}_t = f(x_t; \Theta_t)$ using parameters Θ_t. For gradient-based decision trees, Θ_t consists of feature weight matrix $\mathbf{I}_t \in \mathbb{R}^{(2^d-1)\times n_t}$, threshold matrix $\mathbf{T}_t \in \mathbb{R}^{(2^d-1)\times n_t}$, and leaf prediction matrix $\mathbf{L}_t \in \mathbb{R}^{2^d \times C}$, where d is the tree depth fixed at initialization. The learning objective is to find parameters Θ_t that min-

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

imize the expected classification loss over the stream:

$$\min_{\Theta_t} \mathbb{E}_{(x_t,y_t)\sim\mathcal{S}} \left[\ell(y_t, \hat{y}_t)\right] \tag{3.1}$$

where ℓ is the classification loss function. This objective must be achieved while adapting to both the growth of n_t and the variation in F_t^{obs} across samples.

3.2 Gradient-Based Tree Optimization Traditional decision trees select split features through discrete optimization at each node, which blocks gradient flow during backpropagation. Recent work has addressed this limitation by representing tree parameters as differentiable matrices that can be optimized via gradient descent (40). We build upon this parametrization framework. A decision tree of depth d is represented using three parameter matrices. Feature weight $\mathbf{I} \in \mathbb{R}^{(2^d-1)\times n}$ assigns a learnable weight to each of n features at each of the $(2^d - 1)$ internal nodes, where higher weights indicate stronger preference for selecting that feature. Threshold $\mathbf{T} \in \mathbb{R}^{(2^d-1)\times n}$ stores split thresholds for each feature at each node. Leaf prediction $\mathbf{L} \in \mathbb{R}^{2^d\times C}$ provides class probability at the 2^d leaf nodes. All three matrices are jointly optimized through gradient descent to minimize classification loss. This parametrization enables end-to-end learning of tree decisions, but assumes a fixed feature space where n is known before training and all features are available for every sample. We present in the following section how DynaGradTree adapts this framework to VFS scenarios.

4 DynaGradTree We propose DynaGradTree for online tree learning in VFS scenarios. The core challenge is twofold. First, the feature space itself grows as new features emerge in the data stream, requiring the model to accommodate features not present during initialization. Second, individual samples may lack arbitrary subsets of features, requiring adaptive routing that selects different split features for different samples based on feature availability. Our key insight is that soft feature selection naturally enables learned surrogate splits when combined with instance-level masking. By parametrizing tree decisions using differentiable matrices, the soft feature selection at each node learns a ranking of features based on their splitting quality. When a sample lacks the optimal feature, instance-level masking suppresses unavailable features in the weight distribution, causing the model to automatically select the best available alternative. This implements learned surrogate splits through end-to-end gradient optimization rather than post-hoc construction.

To enable this in online VFS scenarios, we address two technical challenges. When new features appear in the stream, we dynamically initialize their parameters in the weight and threshold matrices using data-driven strategies. This allows newly arrived features to immediately participate in gradient-based optimization without requiring model reconstruction. For samples with limited feature availability, we apply instance-level masking to modulate the soft feature selection at each node based on which features are present. The masking mechanism integrates seamlessly with the gradient-based optimization framework, enabling sample-adaptive routing while maintaining differentiability for parameter updates.

The resulting framework operates as follows. DynaGradTree maintains a complete binary tree of fixed depth d determined at initialization. Parameters consist of feature weight matrix $\mathbf{I}_t \in \mathbb{R}^{(2^d-1)\times n_t}$, threshold matrix $\mathbf{T}_t \in \mathbb{R}^{(2^d-1)\times n_t}$, and leaf prediction matrix $\mathbf{L}_t \in \mathbb{R}^{2^d\times C}$, where n_t is the number of features observed up to time t. As the stream progresses, n_t grows when new features arrive, triggering parameter initialization. For each incoming sample, instance-level masking produces sample-specific routing decisions based on feature availability. All parameters are updated through mini-batch gradient descent, enabling the model to learn from streaming data while handling both feature space growth and varying feature availability across samples.

4.1 Dynamic Parameter Initialization for Feature Space Growth In online learning with VFS, new features continuously emerge as the data stream progresses. The model must accommodate these new features while preserving previously learned decision structures. Since the parameter matrices $\mathbf{I} \in \mathbb{R}^{(2^d-1)\times n}$ and $\mathbf{T} \in \mathbb{R}^{(2^d-1)\times n}$ encode feature weights and thresholds for all nodes, their dimensionality must grow to incorporate newly arrived features.

When a new feature f_{new} appears, the model dynamically expand the parameter matrices by adding new columns and initializing the corresponding parameters. For each internal node i, we initialize the feature weight as $\mathbf{I}[i, f_{\text{new}}] = \frac{1}{n_t}\sum_{j=1}^{n_t} \mathbf{I}[i, j]$ and the threshold as $\mathbf{T}[i, f_{\text{new}}] = \frac{1}{|\mathcal{B}_t|}\sum_{x\in\mathcal{B}_t} x[f_{\text{new}}]$, where n_t is the number of features observed up to time t, and $\mathcal{B}_t$ is the current mini-batch containing f_{new}. The feature weight initialization sets the importance of new feature to the average of existing features at each node. This neutral starting point ensures the new feature neither dominates nor is ignored during feature selection, allowing gradient descent to adjust its weight as more data arrives. The threshold initialization uses the mean value of f_{new} in its first appearance, providing a reasonable starting point within the value range of the feature.

At time t, the parameter matrices have dimension

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

$(2^d - 1) \times n_t$, where n_t grows as new features are encountered. This dynamic growth ensures that the model capacity expands organically with the observed feature space, with each newly initialized parameter immediately participating in gradient-based optimization.

4.2 Instance-Level Masking for Surrogate Splits In gradient-based tree learning, feature weight matrix $\mathbf{I}$ at each node undergoes entmax (43) transformation (detailed in Appendix I) followed by hardmax to get a one-hot vector selecting a single split feature. While this identifies the optimal feature for splitting, it overlooks valuable information in the complete weight distribution. Each row $\mathbf{I}_i$ represents learned preferences over all features for splitting at node i, with higher weights indicating greater suitability. For samples where the optimal feature is unavailable, the weight distribution naturally encodes a ranking of alternative features that could serve as substitutes. Therefore, we leverage this observation through instance-level masking, enabling the model to select different split features for different samples based on feature availability. For each sample x during forward propagation, we construct an instance-level mask $\mathbf{M}_x \in \mathbb{R}^{n_t}$ where each element $\mathbf{M}_x[j]$ equals 0 if feature j is present in x and equals $-K$ (we use $K = 1000$ throughout) if feature $j \notin x$. Before computing split decisions at each node i, we apply this mask to the feature weights through $\mathbf{I}_i^x = \mathbf{I}_i + \mathbf{M}_x$, ensuring that positions corresponding to unavailable features receive large negative values. Then the entmax and hardmax transformations select the highest-weighted feature among those actually present in sample x. Figure 4.1 illustrates this process with an example showing how two samples with different feature availability are routed through the same node. This design implements learned surrogate splits without requiring explicit surrogate construction. Unlike post-hoc local optimization at each node, this approach learns feature preferences jointly through end-to-end gradient descent. The weight distribution in $\mathbf{I}_i$ is learned from global optimization and encodes not only the primary split feature but also the relative suitability of all features. When the primary feature of a sample is absent, the instance-level mask allows the model to select the most appropriate substitute based on the learned ranking.

To handle samples with varying feature availability, we extend each sample $x \in \mathbb{R}^{n_x}$ to dimension n_t by padding unobserved positions with zeros, where n_x denotes the number of features present. This prevents NaN propagation while the masking mechanism ensures these positions cannot be selected. The one-hot feature selection for sample x at node i is computed as $\mathbf{I}^x_{\text{onehot},i} = \text{entmax}(\mathbf{I}_i^x) - ST(\text{entmax}(\mathbf{I}_i^x) - \text{hardmax}(\text{entmax}(\mathbf{I}_i^x)))$, where entmax (43) produces a sparse probability distribution over features and the straight-through operator ST (3) enables gradient flow. In the forward pass, this produces a discrete one-hot vector selecting a single feature. In the backward pass, gradients flow through the continuous entmax output. Using the selected feature, the split decision at node i for sample x is computed as:

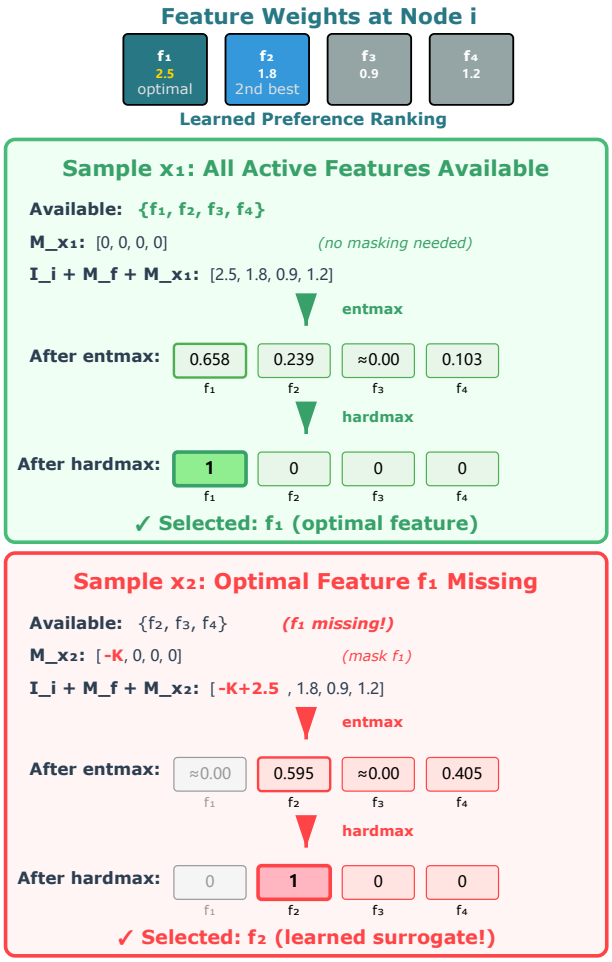


Figure 4.1: Instance-level masking for learned surrogate splits. This figure shows the masking mechanism at a node with learned feature weights. Sample x_1 with all features available selects the optimal feature f_1. Sample x_2 lacking f_1 applies instance-level mask $\mathbf{M}_{x_2}$ that filters out f_1, causing the model to automatically select f_2 as the best available alternative based on the learned preference ranking. The surrogate selection is learned from end-to-end gradient optimization.

$$S(x|\mathbf{I}_i^x, \mathbf{T}_i) = \lfloor \sigma(\mathbf{I}^x_{\text{onehot},i} \cdot x - \mathbf{I}^x_{\text{onehot},i} \cdot \mathbf{T}_i) \rceil \tag{4.1}$$

where $\sigma(z) = 1/(1+e^{-z})$ is the logistic function, and $\lfloor \cdot \rceil$ denotes rounding to the nearest integer. The straight-through operator is similarly applied to the rounding

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

operation to maintain gradient flow while producing hard binary decisions in the forward pass.

To determine which leaf a sample x reaches, we compute an indicator function by multiplying split decisions along the path from root to leaf. For a tree of depth d, the indicator function for leaf l is given by $\mathbb{L}(x|l, \mathbf{T}, \mathbf{I}, \mathbf{M}_x) = \prod_{j=1}^{d}[(1-p(l,j))\cdot S(x|\mathbf{I}^x_{i(l,j)}, \mathbf{T}_{i(l,j)})+ p(l,j)\cdot(1-S(x|\mathbf{I}^x_{i(l,j)}, \mathbf{T}_{i(l,j)}))]$, where $i(l,j)$ identifies the internal node at depth j along the path to leaf l, and $p(l,j) \in \{0,1\}$ indicates whether the path takes the left branch ($p=0$) or right branch ($p=1$). The indicator equals 1 if sample x reaches leaf l and 0 otherwise. The final prediction combines contributions from all leaves weighted by their reachability: $g(x|\mathbf{L}, \mathbf{T}, \mathbf{I}, \mathbf{M}_x) = \sum_{l=0}^{2^d-1} \mathbf{L}_l \cdot \mathbb{L}(x|l, \mathbf{T}, \mathbf{I}, \mathbf{M}_x)$, where $\mathbf{L}_l \in \mathbb{R}^C$ contains the class logits at leaf l. Although this is written as a sum over all 2^d leaves for compact notation, the straight-through rounding applied to each $S(x|\cdot)$ (Eq. 4.1) ensures $\mathbb{L}(x|l,\cdot) \in \{0,1\}$ with exactly one leaf satisfying $\mathbb{L}(x|l,\cdot)=1$ per sample. Each prediction therefore corresponds to a single, hard decision path, structurally identical to a standard axis-aligned decision tree, rather than a soft mixture over leaves. For classification, the predicted class is $\hat{y} = \arg\max_c g(x)_c$. For training, we minimize the cross-entropy loss between predicted logits $g(x)$ and true label y.

This formulation shows how instance-level masking integrates into the gradient-based tree framework. Each sample follows a potentially different decision path based on its feature availability, with the masking mechanism ensuring that unavailable features cannot be selected at any node. All parameters $\mathbf{I}$, $\mathbf{T}$, and $\mathbf{L}$ are jointly optimized through backpropagation, enabling the model to learn feature preferences that account for varying feature availability patterns. The complete online learning procedure that integrates these mechanisms is presented in Algorithm A.1 in Appendix A.

4.3 Complexity Analysis DynaGradTree maintains parameter matrices of total size $O(2^d \cdot n_t)$. Processing T samples requires $O(T \cdot 2^d \cdot n_t)$ time, which is linear in both the number of samples and the feature dimension since d is predetermined. Detailed derivations are provided in Appendix B.

5 Experiments

5.1 Experimental Setup We evaluate DynaGradTree through comprehensive experiments comprising quantitative comparisons against baselines and qualitative validation of the learned surrogate mechanism.

5.1.1 Main Experiments We conduct two comprehensive experiments to evaluate the performance of DynaGradTree against traditional online decision trees and VFS-specific methods.

Datasets. We evaluate on 14 real-world datasets in different domains and complexity levels (detailed in Appendix C). Each dataset is processed in 100 mini-batches using up to 10,000 samples where available.

Evaluation. We employ prequential evaluation (test-then-train). Each experiment is repeated 10 times with different random seeds, and we report mean and standard deviation of Cumulative Error Rate (CER).

Experimental Design. We conduct two experiments evaluating varying feature availability: one in fixed feature spaces, the other in growing feature spaces.

Baselines. We compare DynaGradTree against two categories of methods. For traditional online trees, we evaluate against Hoeffding Tree, EFDT, and HAT using a trapezoidal absence pattern where feature availability improves from 70% to 100%. For VFS-specific methods, we compare against OLVF, OLSF, ORF3V, and OVFM using both feature growth pattern (10% to 100% of all features) and the same feature availability pattern.

5.1.2 Qualitative Study For in-depth analysis of the learned surrogate mechanism, we construct an experimental environment using a synthetic dataset with three complementary analyses: ablation study, surrogate feature testing, and consistency evaluation.

Dataset Construction. We use a synthetic dataset with 8,000 samples and 20 features organized into 3 correlated groups. Features 0-5 form the groups with pairwise correlation of 0.85, while the remaining 14 features are independent. Correlated groups have higher importance weights, enabling precise experimental control.

Experimental Configurations. For ablation study, we design four feature availability patterns (Arbitrarily Varying, Feature Absence without Recurring, Trapezoidal Feature Absence, Feature Absence with Recurring) at three absence ratios (10%, 20%, 30%), yielding 12 configurations. We compare learned surrogate mechanism with Zero-Filling and Mean-Filling baselines. For surrogate feature testing, we design four scenarios based on which features are unavailable: Primary Features Absent (features 0, 2, 4), Surrogate Features Absent (features 1, 3, 5), Cross Absent (features 0, 3, 4), and High Random Absence (50% random). Each scenario is evaluated on 2,000 test samples. For consistency evaluation, we generate 10 random absence patterns per test sample at absence ratios from 5% to 30%, comparing with Hoeffding Tree with mean-filling and OLVF to assess decision stability under varying absence ratios.

5.2 Experimental Results We document all experimental results in Tables 5.1–5.4, enabling us to answer the following Research Questions (RQ1–RQ4).

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

RQ1: Does DynaGradTree achieve competitive or better performance compared to traditional online decision tree methods? If so, under what conditions does it excel?

DynaGradTree demonstrates better performance, achieving 1st or 2nd place on 11 out of 14 datasets when compared against other online tree model baselines. Specifically, it ranks 1st on 7 datasets and 2nd on 4 datasets. Performance analysis shows that DynaGradTree excels particularly on complex datasets with higher dimensionality or larger numbers of classes, like Mushroom, Nomao, and Satimage, where the learned surrogate mechanism effectively handles rich feature spaces. Conversely, traditional methods perform better on simpler datasets like Adult and Airline, where simple decision boundaries reduce the relative value of adaptive feature selection. The learned surrogate mechanism provides the most significant benefits in scenarios demanding sophisticated feature space navigation. While traditional online trees are limited to fixed feature spaces, these results demonstrate the competitiveness of DynaGradTree in this constrained setting while uniquely having the capability to handle VFS, which is a main advantage unavailable to the baselines. Learning curves for all datasets are provided in Appendix E.

RQ2: Does DynaGradTree achieve competitive or better performance compared to VFS-specific baselines? If so, under what conditions does it excel?

DynaGradTree demonstrates competitive performance against specialized VFS methods, achieving 1st or 2nd place on 8 out of 14 datasets when compared with the baselines. Specifically, it ranks 1st on 6 datasets and 2nd on 2 datasets. The complexity-dependent pattern observed with traditional trees extends to VFS scenarios: DynaGradTree shows strong performance on datasets like Adult, Nomao, Electricity, and Satimage. On datasets where other methods outperform DynaGradTree, dataset characteristics provide important context. Methods like OVFM achieve better results on Covertype, Phishing, and Gas Sensor, which contain mixed feature types combining numerical, binary, and categorical features. OVFM was specifically designed for mixed-type scenarios through latent representations, while DynaGradTree targets the VFS challenge of dynamic feature spaces and varying availability. Notably, on high-dimensional datasets like Nomao and Gas Sensor, DynaGradTree achieves competitive results, suggesting the learned surrogate mechanism effectively handles feature-rich environments.

Beyond Accuracy: Interpretability and Efficiency. While OVFM achieves better accuracy on mixed-type datasets, DynaGradTree offers advantages in interpretability and runtime efficiency. The tree structure provides clear decision rules that can be understood and validated, unlike latent representations of OVFM. However, detailed runtime analysis (**RQ4**) shows that the accuracy gains of OVFM come at substantial computational cost, especially on high-dimensional data. DynaGradTree, as a single-tree approach, bridges the gap between traditional online trees and VFS-specific methods while maintaining practical applicability. Learning curves are shown in Appendix F.

RQ3: How effective is learned surrogate mechanism in handling varying feature availability?

To enable in-depth analysis of the learned surrogate mechanism, we conducted controlled experiments using the synthetic dataset described in 5.1.2. We validate mechanism effectiveness through three complementary analyses: ablation studies comparing learned surrogates against imputation baselines, surrogate feature testing examining whether the model learns feature substitution patterns, and consistency analysis evaluating decision stability under varying absence patterns.

Ablation Study Table 5.3 presents results across all 12 experimental configurations, Surrogate Learning consistently outperforms both baselines with 3-4% relative improvements on average across all patterns and absence ratios. Notably, Mean-Filling shows no consistent improvement over Zero-Filling, indicating that simple imputation cannot leverage feature correlations effectively. In contrast, the learned surrogate mechanism achieves better performance through feature substitution strategies, showing that it provides genuine value beyond simple imputation. Curves for all configurations are in Appendix G.

Surrogate Feature Testing Beyond performance gains, we examined whether the model learns feature substitution patterns rather than general improvements. Using four scenarios based on which correlated features are unavailable, Table 5.4 reveals a symmetric pattern that evidences interpretability. In Primary Absent, Surrogate Learning outperforms Mean-Filling by 1.25%, validating learned surrogate usage when primaries are unavailable. For Surrogates Absent, this advantage disappears, with performance nearly identical to Mean-Filling (0.70% difference). This symmetry demonstrates that the model learns to use surrogates when primaries are unavailable rather than becoming better overall. Cross Absent and High Absent demonstrate robustness, with Surrogate Learning performing better in 7 out of 8 comparisons. To verify structural stability, we repeated experiments with five random shuffles of training data. Primary features and surrogates consistently appeared among the

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

Table 5.1: Performance comparison with traditional online decision trees. We report CER (mean ± std) over 10 runs, the rank of DynaGradTree among 4 methods, and average runtime. Runtime analysis is detailed in **RQ4**. Best results in **bold**, second-best underlined.

Dataset	DynaGradTree	Hoeffding Tree	EFDT	HAT	Rank
Covertype	**0.4804±0.0103**	0.5139±0.0112	0.5080±0.0076	0.5141±0.0149	1/4
Electricity	**0.3034±0.0094**	0.3235±0.0198	0.3116±0.0061	0.3236±0.0127	1/4
Mushroom	**0.1056±0.0135**	0.3179±0.0172	0.1329±0.0108	0.2273±0.0089	1/4
Nomao	**0.1509±0.0157**	0.2093±0.0069	0.1567±0.0191	0.1741±0.0114	1/4
Phishing	**0.1063±0.0078**	0.1590±0.0143	0.1124±0.0165	0.1572±0.0092	1/4
Poker	**0.4447±0.0121**	0.4489±0.0187	0.4998±0.0054	0.4668±0.0139	1/4
Satimage	**0.2587±0.0103**	0.7637±0.0156	0.3253±0.0178	0.7675±0.0067	1/4
Gas Sensor	0.7344±0.0194	0.7834±0.0081	**0.4527±0.0146**	0.7859±0.0123	2/4
Letter	0.8409±0.0065	0.9616±0.0132	**0.7875±0.0189**	0.9608±0.0101	2/4
Pendigits	0.5216±0.0117	0.9015±0.0073	**0.4176±0.0152**	0.8991±0.0196	2/4
Shuttle	0.2040±0.0141	0.2171±0.0109	**0.1592±0.0068**	0.1823±0.0174	3/4
Wall Robot	0.4954±0.0059	0.5988±0.0168	**0.4177±0.0137**	0.5992±0.0098	2/4
Adult	0.2578±0.0180	0.2362±0.0125	0.2352±0.0071	**0.2304±0.0159**	4/4
Airline	0.4756±0.0090	0.4359±0.0192	**0.4341±0.0116**	0.4415±0.0063	4/4
Avg Runtime (s)	75	**8**	87	11	–
Rank distribution: 1st place (7), 2nd place (4), 3rd place (1), 4th place (2)					

top-3 ranked splitting candidates in the top 3 layers with over 90% frequency, demonstrating that learned patterns converge regardless of data arrival order. Tree visualization is provided in Appendix H.

Consistency Analysis To validate decision stability, we conducted a prediction consistency test where each test sample was evaluated under 10 random absence patterns at absence ratios from 5% to 30%. Each pattern maintains the same absence ratio but varies which specific features are unavailable, allowing us to quantify how often predictions match those on complete data. Figure 5.1 shows that DynaGradTree achieves higher consistency than OLVF across all absence rates, and than Hoeffding Tree with mean-filling at absence rates above 10%. This advantage stems from different handling strategies: OLVF maintains separate model states for different feature subsets, causing inconsistent predictions when absence patterns change, while DynaGradTree learns to select the best available surrogate, resulting in more stable decisions.

Together, these three analyses demonstrate that the learned surrogate mechanism not only improves performance but does so through interpretable patterns and stable decision-making strategies.

RQ4: How efficient is DynaGradTree in terms of computational performance?

To evaluate practical applicability, we conducted runtime analysis across all experimental configurations. Tables 5.1 and 5.2 include average runtime measure-

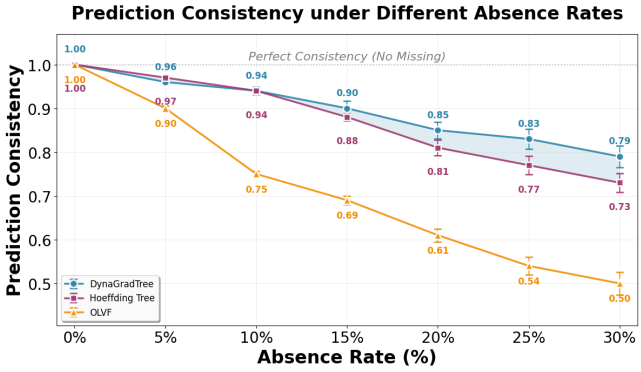


Figure 5.1: Prediction consistency vs absence rates.

ments, with detailed results provided in Appendix D.

Findings: Runtime analysis reveals that DynaGradTree achieves practical efficiency for real-time streaming while validating our theoretical complexity analysis. Three key findings highlight computational trade-offs.

Practical Performance of DynaGradTree. DynaGradTree averages 75s total wall-clock time in the traditional online tree comparison and 70s in the VFS method comparison, measured across the full stream (100 mini-batches, 5,000-10,000 samples per dataset). Dividing total time by sample count, per-sample processing ranges from 1.7 to 22.3 ms across all 14 datasets (mean 7.7 ms). For streaming applications where samples arrive at rates of 10-100 per second, this processing

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

Table 5.2: Performance comparison with VFS-specific methods. We report CER (mean ± std) over 10 runs, the rank of DynaGradTree among 5 methods, and average runtime. Runtime analysis is detailed in **RQ4**. Best results in **bold**, second-best underlined.

Dataset	DynaGradTree	OLVF	OLSF	ORF3V	OVFM	Rank
Covertype	0.5057±0.0184	0.5169±0.0271	0.5627±0.0229	0.5152±0.0148	**0.3773±0.0197**	2/5
Electricity	**0.3742±0.0159**	0.4515±0.0213	0.4917±0.0176	0.4255±0.0294	0.4096±0.0138	1/5
Mushroom	0.1892±0.0241	0.2726±0.0187	0.3803±0.0268	**0.1558±0.0119**	0.1765±0.0225	3/5
Nomao	**0.1461±0.0276**	0.1545±0.0142	0.1931±0.0194	0.2858±0.0157	0.2027±0.0281	1/5
Phishing	0.1813±0.0128	0.2456±0.0222	0.4521±0.0179	0.3485±0.0254	**0.1366±0.0183**	2/5
Poker	**0.4882±0.0216**	0.5036±0.0198	0.5054±0.0287	0.4969±0.0145	0.4991±0.0273	1/5
Satimage	**0.2450±0.0174**	0.7679±0.0239	0.8020±0.0158	0.3793±0.0211	0.2491±0.0290	1/5
Gas Sensor	0.7632±0.0137	0.7713±0.0248	0.7698±0.0186	0.6819±0.0272	**0.3191±0.0169**	3/5
Letter	0.8996±0.0255	0.9594±0.0171	0.9610±0.0144	0.8555±0.0227	**0.8145±0.0198**	3/5
Pendigits	0.7459±0.0118	0.8577±0.0263	0.8635±0.0209	0.5198±0.0152	**0.4562±0.0284**	3/5
Shuttle	0.4960±0.0221	0.4753±0.0188	0.3679±0.0149	0.2135±0.0277	**0.1467±0.0165**	5/5
Wall Robot	0.5582±0.0196	0.5541±0.0282	0.5967±0.0173	0.5309±0.0234	**0.4568±0.0147**	4/5
Adult	**0.1811±0.0129**	0.3150±0.0251	0.4305±0.0143	0.2367±0.0218	0.1902±0.0175	1/5
Airline	**0.4412±0.0168**	0.4822±0.0184	0.4962±0.0279	0.4459±0.0156	0.4455±0.0231	1/5
Avg Runtime (s)	70	**9**	9	43	183	–
Rank distribution: 1st place (6), 2nd place (2), 3rd place (4), 4th place (1), 5th place (1)						

Table 5.3: Ablation study results: Final CER and absolute improvements (in percentage points). Negative Δ values indicate Surrogate performs better.

absence ratio	**Surr.**	**Zero**	**Mean**	**ΔZ**	**ΔM**
Feature Space Arbitrarily Varying					
10%	0.4928	0.5238	0.5342	-3.10	-4.14
20%	0.5095	0.5486	0.5524	-3.91	-4.29
30%	0.5228	0.5599	0.5601	-3.71	-3.73
Feature Absence w/o Recurring					
10%	0.5119	0.5189	0.5153	-0.70	-0.34
20%	0.4951	0.5215	0.5157	-2.64	-2.06
30%	0.5030	0.5073	0.5059	-0.43	-0.29
Trapezoidal Feature Absence					
10%	0.4954	0.5081	0.5061	-1.27	-1.07
20%	0.5005	0.5130	0.5168	-1.25	-1.63
30%	0.4951	0.5309	0.5137	-3.58	-1.86
Feature Absence w/ Recurring					
10%	0.4971	0.5163	0.5024	-1.92	-0.53
20%	0.4927	0.5044	0.5116	-1.17	-1.89
30%	0.4908	0.5067	0.5065	-1.59	-1.57

Table 5.4: Surrogate feature testing results: Accuracy and absolute improvements (in percentage points). Negative Δ values indicate Surrogate performs better.

Scenario	**Surr.**	**Zero**	**Mean**	**ΔZ**	**ΔM**
Primary Absent	0.4895	0.4365	0.4770	-5.30	-1.25
Surrogate Absent	0.4970	0.4430	0.5040	-5.40	+0.70
Cross Absent	0.4950	0.3295	0.4770	-16.55	-1.80
High Absent (50%)	0.4710	0.4275	0.4115	-4.35	-5.95

time allows real-time model updates without accumulating backlog. Importantly, the runtime validates our theoretical complexity analysis (Subsection 4.3): the observed correlation between runtime and the product of sample count and feature count confirms the $O(T \cdot n_t)$ linear scaling of our analysis.

Tree Growth Bottleneck of EFDT. EFDT exhibits high variance due to unconstrained growth. On datasets with frequent splitting, structures become extremely large, creating a paradox that "extremely fast" growth produces slow traversal. Appendix D shows EFDT requires over 250 seconds on Gas Sensor and Letter, substantially longer than DynaGradTree.

High-Dimensional Bottleneck of OVFM. The most striking finding concerns the scalability of OVFM. Although OVFM achieves competitive or better accuracy on some datasets (Table 5.2), its average runtime of 183s shows problems on high-dimensional data. Detailed results show OVFM requires 1479s on Gas Sensor and 791s on Nomao, which is 3.5 to 7.4 times slower than DynaGradTree. The high variance indicates highly unpredictable running time that makes OVFM unsuitable for production deployments for VFS.

These results demonstrate that DynaGradTree achieves competitive accuracy while maintaining consistent, practical runtime across diverse scenarios. Unlike the fast tree growth of EFDT or high-dimensional collapse of OVFM, DynaGradTree provides reliable performance suitable for real-world streaming applications.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

6 Conclusion and Future Work We introduced DynaGradTree, a tree method for streaming data that handles VFS and also preserves interpretable decision structures. Through learned surrogate splits, DynaGradTree automatically selects the most informative available features for each sample based on feature availability without static imputation. Experiments across 14 datasets demonstrate competitive performance against both traditional online trees and VFS-specific methods. Ablation studies validate that performance improvements stem from learned substitution strategies, and surrogate feature testing confirms interpretable mechanism behavior. Runtime analysis shows practical efficiency suitable for streaming applications.

Several directions remain open for future research. First, the forward pass complexity $O(2^d \cdot n_t)$ grows with tree depth. An important next step is exploring adaptive-depth mechanisms that can fit very high-dimensional streams. Second, currently we adopted entmax with $\alpha = 1.5$ for soft feature selection; investigating alternative sparse selection mechanisms and their trade-offs between sparsity and gradient smoothness under rapidly changing feature availability is a promising direction for performance improvement. Third, dynamic parameter initialization for newly arriving features currently relies on a mean-based heuristic; developing alternative initialization strategies could further improve stability as the feature space evolves non-monotonically. Gradient-based online decision trees remain a relatively young paradigm, with few existing methods to compare against (33); our ablation studies and surrogate feature testing provide an initial empirical foundation that we hope will support further investigation of this direction. Finally, extending validation to real-world datasets that naturally exhibit the addressed missingness patterns, and to scenarios combining concept drift with feature space evolution, would further strengthen the practical case for learned surrogate splits in streaming settings.

Acknowledgment This work was supported in part by the National Science Foundation (NSF) under Grant Nos. IIS-2544353 (CAREER), IIS-2505719, IIS-2236579, IIS-2302786, IIS-2441449, IOS-2430224, and IOS-2446522, the Science Center for Marine Fisheries, and the Commonwealth Cyber Initiative (CCI).

References

[1] D. N. Assis, J. P. Barddal, and F. Enembreck, *Behavioral insights of adaptive splitting decision trees in evolving data stream classification*, Knowl. Inf. Syst., 67 (2025), pp. 5751–5782.

[2] Z. Azam, M. M. Islam, and M. N. Huda, *Comparative analysis of intrusion detection systems and machine learning-based model analysis through decision tree*, IEEE Access, 11 (2023), pp. 80348–80391.

[3] Y. Bengio, N. Léonard, and A. Courville, *Estimating or propagating gradients through stochastic neurons for conditional computation*, arXiv preprint arXiv:1308.3432, (2013).

[4] E. Beyazit, J. Alagurajah, and X. Wu, *Online learning from data streams with varying feature spaces*, in Proc. of the AAAI Conf. on Artificial Intelligence, vol. 33, 2019, pp. 3232–3239.

[5] A. Bifet, E. Frank, G. Holmes, and B. Pfahringer, *Ensembles of restricted hoeffding trees*, in ACM Trans. Intell. Syst. Technol., vol. 3, 2012, pp. 30:1–30:20.

[6] A. Bifet and R. Gavaldà, *Adaptive learning from evolving data streams*, in Proc. 11th Int. Symp. Intell. Data Anal., Dresden, Germany, 2009, pp. 249–260.

[7] A. Bifet, G. Holmes, B. Pfahringer, and E. Frank, *Fast perceptron decision tree learning from evolving data streams*, in Proc. Pacific-Asia Conf. Knowl. Discovery Data Mining, Hyderabad, India, 2010, pp. 299–310.

[8] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*, Wadsworth International Group, Belmont, CA, USA, 1984.

[9] L. Candillier and V. Lemaire, *Design and analysis of the Nomao challenge: Active learning in the real-world*, in Workshop on Active Learning in Real-World Applications, ECML-PKDD, 2012.

[10] V. G. Costa and C. E. Pedreira, *Recent advances in decision trees: An updated survey*, Artificial Intel. Rev., 56 (2023), pp. 4765–4800.

[11] P. Domingos and G. Hulten, *Mining high-speed data streams*, in Proc. 6th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, Boston, MA, USA, 2000, pp. 71–80.

[12] D. Dua and C. Graff, *UCI machine learning repository*. http://archive.ics.uci.edu/ml, 2017.

[13] N. Frosst and G. Hinton, *Distilling a neural network into a soft decision tree*, in Proceedings of the First International Workshop on Comprehensibility and Explanation in AI and ML, 2017.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

[14] J. Gama, R. Rocha, and P. Medas, *Accurate decision trees for mining high-speed data streams*, in Proc. 9th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, Washington, DC, USA, 2003, pp. 523–528.

[15] H. M. Gomes, A. Bifet, J. Read, J. P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, and T. Abdessalem, *Adaptive random forests for evolving data stream classification*, Mach. Learn., 106 (2017), pp. 1469–1495.

[16] H. M. Gomes, J. Read, and A. Bifet, *Streaming random patches for evolving data stream classification*, in Proc. IEEE Int. Conf. Data Mining, Beijing, China, 2019, pp. 240–249.

[17] J. Gong, *Decision trees in business and finance: A review of applications, challenges, and future directions*, Advances in Economics and Management Research, 15 (2025), pp. 683–683.

[18] N. Gunasekara, B. Pfahringer, H. M. Gomes, and A. Bifet, *Gradient boosted trees for evolving data streams*, Mach. Learn., 113 (2024), pp. 3325–3352.

[19] M. Harries, *Splice-2 comparative evaluation: Electricity pricing*, technical report, The University of New South Wales, Sydney, Australia, 1999.

[20] V. Hassija, V. Chamola, A. Mahapatra, A. Singal, D. Goel, K. Huang, S. Scardapane, I. Spinelli, M. Mahmud, and A. Hussain, *Interpreting black-box models: a review on explainable artificial intelligence*, Cognitive Computation, 16 (2024), pp. 45–74.

[21] Y. He, J. Dong, B.-J. Hou, Y. Wang, and F. Wang, *Online learning in variable feature spaces with mixed data*, in 2021 IEEE International Conference on Data Mining (ICDM), IEEE, 2021, pp. 181–190.

[22] Y. He, C. Schreckenberger, H. Stuckenschmidt, and X. Wu, *Towards utilitarian online learning-a review of online algorithms in open feature space*, in Proc. 32nd Int. Joint Conf. Artif. Intell., 2023, pp. 6647–6655.

[23] Y. He, B. Wu, D. Wu, E. Beyazit, S. Chen, and X. Wu, *Online learning from capricious data streams: a generative approach*, in Proc. of the Twenty-Eighth Intl. Joint Conf. on Artificial Intelligence (IJCAI), 2019, pp. 2491–2497.

[24] Y. He, B. Wu, D. Wu, E. Beyazit, S. Chen, and X. Wu, *Toward mining capricious data streams: A generative approach*, IEEE Transactions on Neural Networks and Learning Systems, 32 (2020), pp. 1228–1240.

[25] Y. He, X. Yuan, S. Chen, and X. Wu, *Online learning in variable feature spaces under incomplete supervision*, in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, 2021, pp. 4106–4114.

[26] M. Heyden, H. M. Gomes, É. Fouché, B. Pfahringer, and K. Böhm, *Leveraging plasticity in incremental decision trees*, in Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases, 2024, pp. 38–54.

[27] W. Hoeffding, *Probability inequalities for sums of bounded random variables*, in The Collected Works of Wassily Hoeffding, N. I. Fisher and P. K. Sen, eds., Springer, New York, NY, USA, 1994, pp. 409–426.

[28] B.-J. Hou, L. Zhang, and Z.-H. Zhou, *Learning with feature evolvable streams*, Advances in Neural Information Processing Systems, 30 (2017).

[29] C. Hu, Y. Chen, X. Peng, H. Yu, C. Gao, and L. Hu, *A novel feature incremental learning method for sensor-based activity recognition*, IEEE Transactions on Knowledge and Data Engineering, 31 (2018), pp. 1038–1050.

[30] G. Hulten, L. Spencer, and P. Domingos, *Mining time-changing data streams*, in Proc. 7th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, San Francisco, CA, USA, 2001, pp. 97–106.

[31] E. Ikonomovska, J. Gama, and S. Džeroski, *Learning model trees from evolving data streams*, Data Mining and Knowledge Discovery, 23 (2011), pp. 128–168, https://doi.org/10.1007/s10618-010-0201-y.

[32] O. Irsoy, O. T. Yildiz, and E. Alpaydin, *Soft decision trees*, in Proceedings of the 21st International Conference on Pattern Recognition (ICPR), IEEE, 2012, pp. 1819–1822.

[33] K. Köbschall, L. Hartung, and S. Kramer, *Adaptive differentiable trees for transparent learning on data streams*, Mach. Learn., 114 (2025), p. 253.

[34] B. Lakshminarayanan, D. M. Roy, and Y. W. Teh, *Mondrian forests: Efficient online random*

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

forests, Advances in neural information processing systems, 27 (2014).

[35] H. Lian, J. S. Atwood, B.-J. Hou, J. Wu, and Y. He, *Online deep learning from doubly-streaming data*, in Proc. of the 30th ACM Intl. Conf. on Multimedia, 2022, pp. 3185–3194.

[36] H. Lian, D. Wu, B.-J. Hou, J. Wu, and Y. He, *Online learning from evolving feature spaces with deep variational models*, IEEE Trans. on Knowl. & Data Engineering, 36 (2023), pp. 4144–4162.

[37] A. Lourenço, J. Rodrigo, J. Gama, and G. Marreiros, *DFDT: Dynamic fast decision tree for IoT data stream mining on edge devices*, Proc. AAAI Conf. Artif. Intell., 40 (2026), pp. 24053–24060.

[38] C. Manapragada, G. I. Webb, and M. Salehi, *Extremely fast decision tree*, in Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, London, U.K., 2018, pp. 1953–1962.

[39] A. F. T. Martins and R. F. Astudillo, *From softmax to sparsemax: A sparse model of attention and multi-label classification*, in Proc. 33rd Int. Conf. Mach. Learn., New York, NY, USA, 2016, pp. 1614–1623.

[40] S. Marton, S. Lüdtke, C. Bartelt, and H. Stuckenschmidt, *GradTree: Learning axis-aligned decision trees with gradient descent*, in Proc. AAAI Conf. Artif. Intell., vol. 38, Vancouver, BC, Canada, 2024, pp. 14323–14331.

[41] M. M. Masud, Q. Chen, J. Gao, L. Khan, J. Han, and B. Thuraisingham, *Classification and novel class detection of data streams in a dynamic feature space*, in Joint European Conference on Machine Learning and Knowledge Discovery in Databases, Springer, 2010, pp. 337–352.

[42] M. M. Masud, Q. Chen, L. Khan, C. C. Aggarwal, J. Gao, J. Han, A. Srivastava, and N. C. Oza, *Classification and adaptive novel class detection of feature-evolving data streams*, IEEE Transactions on Knowledge and Data Engineering, 25 (2013), pp. 1484–1497.

[43] B. Peters, V. Niculae, and A. F. Martins, *Sparse sequence-to-sequence models*, in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 1504–1519.

[44] V. Podgorelec, P. Kokol, B. Stiglic, and I. Rozman, *Decision trees: an overview and their use in medicine*, Journal of Medical Systems, 26 (2002), pp. 445–463.

[45] S. Popov, S. Morozov, and A. Babenko, *Neural oblivious decision ensembles for deep learning on tabular data*, in International Conference on Learning Representations, 2019.

[46] C. Schreckenberger, C. Bartelt, and H. Stuckenschmidt, *Dynamic forest for learning from data streams with varying feature spaces*, in Intl. conf. on cooperative information systems, Springer, 2022, pp. 95–111.

[47] C. Schreckenberger, T. Glöckner, H. Stuckenschmidt, and C. Bartelt, *Restructuring of Hoeffding trees for trapezoidal data streams*, in Proc. IEEE Int. Conf. Data Mining Workshops, Sorrento, Italy, 2020, pp. 416–423.

[48] C. Schreckenberger, Y. He, S. Lüdtke, C. Bartelt, and H. Stuckenschmidt, *Online random feature forests for learning in varying feature spaces*, in Proc. of the AAAI Conf. on Artificial Intelligence, vol. 37, 2023, pp. 4587–4595.

[49] D. Wu, S. Zhuo, Y. Wang, Z. Chen, and Y. He, *Online semi-supervised learning with mix-typed streaming features*, in Proc. of the AAAI Conf. on Artificial Intelligence, vol. 37, 2023, pp. 4720–4728.

[50] D. You, J. Xiao, Y. Wang, H. Yan, D. Wu, Z. Chen, L. Shen, and X. Wu, *Online learning from incomplete and imbalanced data streams*, IEEE Trans. on Knowl. & Data Engineering, 35 (2023), pp. 10650–10665.

[51] D. You, H. Yan, J. Xiao, Z. Chen, D. Wu, L. Shen, and X. Wu, *Online learning for data streams with incomplete features and labels*, IEEE Transactions on Knowledge and Data Engineering, 36 (2024), pp. 4820–4834.

[52] Q. Zhang, P. Zhang, G. Long, W. Ding, C. Zhang, and X. Wu, *Towards mining trapezoidal data streams*, in 2015 IEEE Intel. Conf. on Data Mining, IEEE, 2015, pp. 1111–1116.

[53] Q. Zhang, P. Zhang, G. Long, W. Ding, C. Zhang, and X. Wu, *Online learning from trapezoidal data streams*, IEEE Tran. on Knowl. & Data Engineering, 28 (2016), pp. 2709–2723.

[54] S. Zhuo, D. Wu, Y. He, S. Huang, and X. Wu, *Online learning from mix-typed, drifted, and incomplete streaming features*, ACM Trans. on Knowl. Discovery from Data, 19 (2025), pp. 1–28.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

A Online Learning Algorithm This appendix provides the complete pseudocode for online learning procedure of DynaGradTree, which integrates the dynamic parameter initialization mechanism (Section 4.1) and instance-level masking mechanism (Section 4.2) with gradient-based parameter updates.

Algorithm A.1 DynaGradTree Online Learning

Require: Tree depth d, initial feature count n_0

Initialize parameter matrices $\mathbf{I} \in \mathbb{R}^{(2^d-1)\times n_0}$, $\mathbf{T} \in \mathbb{R}^{(2^d-1)\times n_0}$, $\mathbf{L} \in \mathbb{R}^{2^d \times C}$ with random values
Initialize $n_t \leftarrow n_0$ to track current feature count
for each mini-batch $\mathcal{B}_t = \{(x_1, y_1), \ldots, (x_m, y_m)\}$ **do**
  // Feature space management
  for each new feature f_{new} detected in $\mathcal{B}_t$ **do**
    Expand $\mathbf{I}$ and $\mathbf{T}$ to dimension $(2^d - 1) \times (n_t + 1)$
    for each node i **do**
      $\mathbf{I}[i, f_{\text{new}}] \leftarrow \frac{1}{n_t} \sum_{j=1}^{n_t} \mathbf{I}[i, j]$
      $\mathbf{T}[i, f_{\text{new}}] \leftarrow \frac{1}{|\mathcal{B}_t|} \sum_{x \in \mathcal{B}_t} x[f_{\text{new}}]$
    end for
    $n_t \leftarrow n_t + 1$
  end for
  // Forward pass with sample-specific masking
  $\mathbf{y} \leftarrow \emptyset$, $\hat{\mathbf{y}} \leftarrow \emptyset$
  for each sample $(x_i, y_i) \in \mathcal{B}_t$ **do**
    Construct instance-level mask $\mathbf{M}_{x_i}$ from absence features in x_i
    Pad x_i to dimension n_t with zeros at absence positions
    for each node j in tree **do**
      Compute masked weights: $\mathbf{I}_j^{x_i} = \mathbf{I}_j + \mathbf{M}_{x_i}$
      Apply entmax + hardmax via ST operator to select feature
      Compute split decision using Equation 4.1
    end for
    Compute prediction $\hat{y}_i$ by following the single hard path determined by split decisions (matrix form used for parallel computation)
    $\mathbf{y} \leftarrow \mathbf{y} \cup y_i$, $\hat{\mathbf{y}} \leftarrow \hat{\mathbf{y}} \cup \hat{y}_i$
  end for
  // Gradient-based parameter update
  Compute classification loss $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$
  // Backpropagation
  Update $\mathbf{I}$, $\mathbf{T}$, and $\mathbf{L}$ using Adam to minimize $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$
end for

The algorithm exhibits two key properties that enable efficient online learning. First, the forward pass for the entire mini-batch can be formulated as a single set of matrix operations rather than iterating over individual samples, which results in efficient parallel computation. Second, the straight-through operator allows gradients to flow through the discrete feature selection and split decisions during backpropagation, enabling continuous optimization of the parameters $\mathbf{I}$ and $\mathbf{T}$ while producing hard, axis-aligned splits in the forward pass. Parameter persistence across mini-batches enables the model to accumulate knowledge incrementally from the entire data stream.

B Detailed Complexity Analysis This appendix provides detailed derivations for the complexity analysis presented in **RQ4** of the main paper.

B.1 Space Complexity Derivation DynaGradTree maintains the following data structures:

Parameter Matrices.

- Feature weight matrix $\mathbf{I} \in \mathbb{R}^{(2^d-1)\times n_t}$: Stores learnable weights over features for each of the $(2^d - 1)$ internal nodes
- Threshold matrix $\mathbf{T} \in \mathbb{R}^{(2^d-1)\times n_t}$: Stores split thresholds for each feature at each internal node
- Leaf prediction matrix $\mathbf{L} \in \mathbb{R}^{2^d \times C}$: Stores class prediction logits for each of the 2^d leaf nodes

The space requirement for these matrices is:

$$S_{\text{params}} = 2 \cdot (2^d - 1) \cdot n_t + 2^d \cdot C \tag{B.1}$$

Mask Vectors.

- Instance-level mask $\mathbf{M}_x \in \mathbb{R}^{n_t}$: Tracks which features are present in the current sample

The space requirement for masks is:

$$S_{\text{masks}} = n_t \tag{B.2}$$

Optimizer State. The Adam optimizer maintains first and second moment estimates for each parameter:

$$S_{\text{optimizer}} = 2 \cdot [2 \cdot (2^d - 1) \cdot n_t + 2^d \cdot C] \tag{B.3}$$

Total Space Complexity. Combining all components:

$$\begin{aligned} S_{\text{total}} &= S_{\text{params}} + S_{\text{masks}} + S_{\text{optimizer}} \\ &= 2(2^d - 1)n_t + 2^d C + n_t \\ &\quad + 4(2^d - 1)n_t + 2 \cdot 2^d C \\ &= 6(2^d - 1)n_t + 3 \cdot 2^d C + n_t \end{aligned} \tag{B.4}$$

$$= O(2^d \cdot n_t + 2^d \cdot C) \tag{B.5}$$

Since d is fixed at initialization and C is typically much smaller than n_t (for example, $C \leq 26$ in our experiments while n_t ranges from 10 to 354), the dominant term is $O(2^d \cdot n_t)$.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

B.2 Time Complexity Derivation We analyze the computational cost for processing a single sample with n_t observed features at round t.

Forward Propagation. At each internal node i along the decision path:

1. Compute combined mask: $\mathbf{I}_i^x = \mathbf{I}_i + \mathbf{M}_f + \mathbf{M}_x$ requires $O(n_t)$ operations (vector addition over n_t non-zero positions)
2. Apply entmax transformation: Computing $\text{entmax}(\mathbf{I}_i^x)$ requires $O(n_t \log n_t)$ operations in the general case, but practical implementations use $O(n_t)$ approximations
3. Feature selection via straight-through operator: $O(1)$ operations after entmax
4. Evaluate split decision: $\lfloor \sigma(\mathbf{I}^x_{\text{onehot},i} \cdot x - \mathbf{I}^x_{\text{onehot},i} \cdot \mathbf{T}_i) \rceil$ requires $O(n_t)$ operations for the dot products

Total cost per node: $O(n_t)$. For a path of length d: $O(d \cdot n_t)$.

Backpropagation. Gradient computation follows the decision path in reverse:

1. Compute loss gradient at leaf: $O(C)$ operations
2. Propagate gradients through d nodes: At each node, compute gradients with respect to $\mathbf{I}_i$ and $\mathbf{T}_i$, requiring $O(n_t)$ operations per node
3. Total backpropagation cost: $O(d \cdot n_t + C)$

Parameter Updates. Adam optimizer updates all parameters that received non-zero gradients:

1. Update $\mathbf{I}$ and $\mathbf{T}$: For each of the $(2^d - 1)$ internal nodes, update the row corresponding to features in the current sample, requiring $O(2^d \cdot n_t)$ operations
2. Update $\mathbf{L}$: Update the leaf reached by the sample, requiring $O(C)$ operations
3. Total update cost: $O(2^d \cdot n_t + C)$

Total Per-Sample Cost.

(B.6)
$$\begin{aligned} T_{\text{sample}} &= T_{\text{forward}} + T_{\text{backward}} + T_{\text{update}} \\ &= O(d \cdot n_t) + O(d \cdot n_t + C) + O(2^d \cdot n_t + C) \\ &= O(2^d \cdot n_t + C) \end{aligned}$$

Since $C \ll n_t$ in most applications, this simplifies to $O(2^d \cdot n_t)$.

Total Stream Cost For T samples: $O(T \cdot 2^d \cdot n_t)$, which is linear in T and n_t with constant factor 2^d.

C Dataset Statistics Table C.1 provides detailed statistics for all 14 datasets used in our experiments, including feature dimensionality, number of classes, and sample sizes.

Table C.1: Summary of 14 real-world datasets used for VFS evaluation.

Dataset	Features	Classes	Samples
Covertype (12)	54	7	10,000
Electricity (19)	8	2	10,000
Mushroom (12)	22	2	8,124
Nomao (9)	118	2	10,000
Phishing (12)	9	2	10,000
Poker (12)	10	10	10,000
Satimage (12)	36	6	6,435
Gas Sensor (12)	128	6	10,000
Letter (12)	16	26	10,000
Pendigits (12)	16	10	10,000
Shuttle (12)	9	7	10,000
Wall Robot (12)	24	4	5,456
Adult (12)	14	2	10,000
Airline (31)	7	2	10,000

D Detailed Runtime Measurements This appendix provides complete runtime measurements for all datasets and methods discussed in Section 5.2 of the main paper.

D.1 Experimental Setup All runtime measurements were conducted on a machine with Intel Core i7-10700K CPU (8 cores, 3.8GHz), 32GB RAM, running Ubuntu 20.04. We disabled GPU acceleration for all methods to ensure fair comparison, as traditional online decision trees (Hoeffding Tree, EFDT, HAT) do not utilize GPUs. Each dataset was processed using the same prequential evaluation protocol described in Section 5 of the main paper, and we report total wall-clock time averaged over 10 runs with different random seeds.

D.2 Detailed Measurements Table D.1 presents runtime for all datasets. The average runtime values shown in the bottom row are summarized in the main text Tables 5.1 and 5.2.

D.3 Key Observations The detailed measurements reveal several patterns discussed in **RQ4**. First, EFDT exhibits extreme variance, with runtime exceeding 250 seconds on Gas Sensor (304s) and Letter (267s) due to unconstrained tree growth. Second, OVFM shows catastrophic slowdown on high-dimensional datasets: 1479 seconds on Gas Sensor (128 features) and 791 seconds on Nomao (118 features), compared to 200 and 223 seconds respectively for DynaGradTree. The standard deviation of OVFM (426 sec-

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

Table D.1: Detailed runtime measurements (in seconds) for all datasets and methods. Each value represents wall-clock time averaged over 10 runs.

Dataset	Traditional Trees				VFS Methods				
	DynaGradTree	HT	EFDT	HAT	DynaGradTree	OLVF	OLSF	ORF3V	OVFM
Adult	156	16	94	18	28	5	6	12	10
Airline	34	9	10	9	20	4	4	6	8
Covertype	57	10	133	15	52	7	7	25	27
Electricity	24	3	5	3	17	4	5	8	13
Gas Sensor	172	15	304	21	200	22	23	187	1479
Letter	76	7	267	16	64	9	8	49	33
Mushroom	138	8	31	11	34	6	6	15	17
Nomao	89	7	105	12	223	28	25	71	791
Pendigits	35	6	79	9	42	8	8	15	17
Phishing	131	10	65	10	46	7	6	19	22
Poker	61	7	21	18	26	5	6	12	17
Satimage	27	5	66	6	42	6	6	14	22
Shuttle	29	4	20	5	148	13	11	152	94
Wall Robot	25	4	21	4	36	6	5	10	12
Mean	75	8	87	11	70	9	9	43	183
Std	53	4	93	6	68	7	7	57	426

onds) exceeding its mean (183 seconds) indicates highly unpredictable performance. Third, simple baselines like Hoeffding Tree and OLVF maintain consistently low runtime across all datasets, though at the cost of limited accuracy as shown in the main text tables.

E Learning Curves: Traditional Trees Comparison This section presents the complete learning curves for all 14 real-world datasets used in the comparison with traditional online decision trees, as reported in Section 5.2 and Table 5.1 of the main paper. The comparison includes three traditional online tree methods: Hoeffding Tree, Extremely Fast Decision Tree (EFDT), and Hoeffding Adaptive Tree (HAT). Each figure shows the Cumulative Error Rate evolution across 100 mini-batches, with mean values and standard deviations computed over 10 independent runs using random seeds 42 through 51. The experimental setup employs the trapezoidal absence pattern where the proportion of absence features gradually decreases from 30% at the beginning to 0% at the final mini-batch.

E.1 Adult Dataset The Adult dataset contains demographic and employment information extracted from census data, with the task of predicting whether an individual's income exceeds fifty thousand dollars per year. With 14 features covering attributes such as age, education level, occupation, and work hours per week, this dataset represents a standard benchmark with moderate dimensionality and balanced class distribution.

E.2 Airline Dataset The Airline dataset consists of flight delay information with the task of predicting whether a flight will be delayed. With 7 features including airline carrier, departure time, day of week, and flight distance, this dataset represents a relatively simple classification problem.

E.3 Covertype Dataset The Covertype dataset contains cartographic features for predicting forest cover types in wilderness areas. With 54 features including elevation, slope, soil type indicators, and distance to water sources, this dataset presents a multi-class classification challenge with 7 different cover types.

E.4 Electricity Dataset The Electricity dataset records electricity pricing in the Australian New South Wales electricity market, with the task of predicting whether prices will increase or decrease. With 8 features including day of week, time of day, and various market indicators, this dataset is known for exhibiting concept drift as electricity pricing patterns change over time.

E.5 Gas Sensor Dataset The Gas Sensor dataset contains measurements from a chemical sensor array exposed to different gas mixtures, with the task of predicting the gas type. With 128 features representing various sensor readings at different time points, this dataset presents a high-dimensional classification problem with 6 gas classes.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

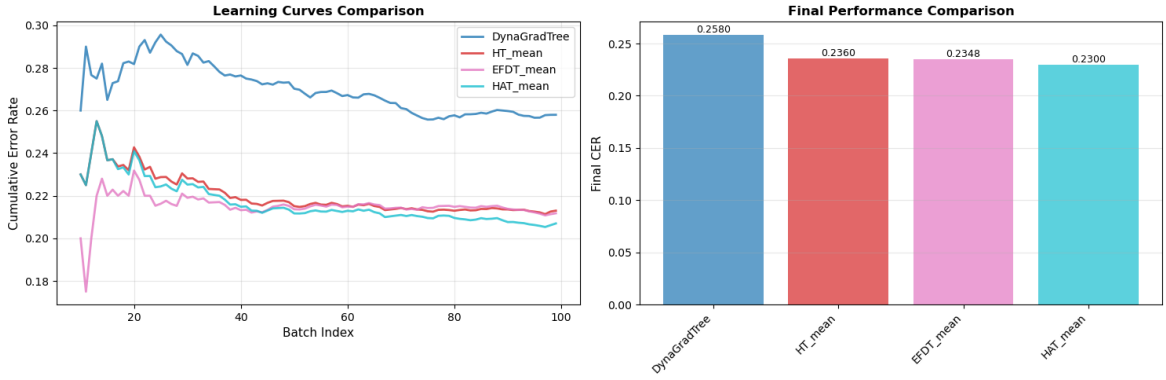


Figure E.1: Learning curve for Adult dataset (14 features, 2 classes, 10,000 samples).

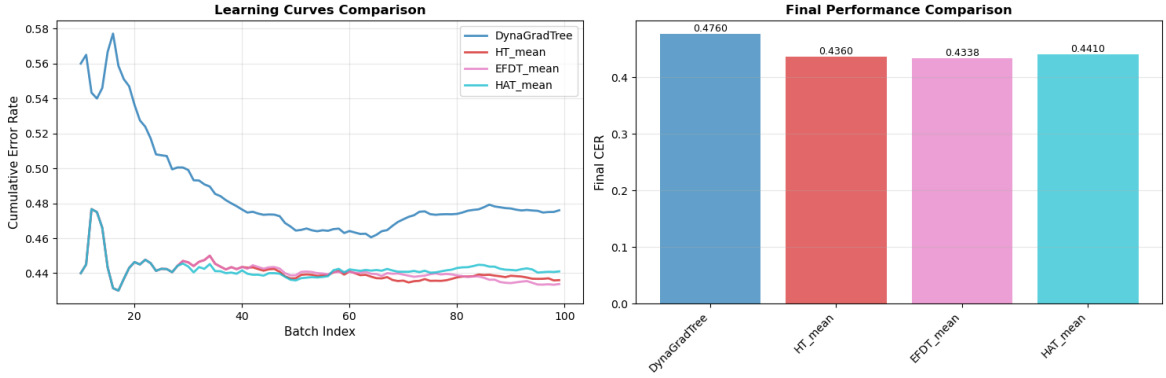


Figure E.2: Learning curve for Airline dataset (7 features, 2 classes, 10,000 samples).

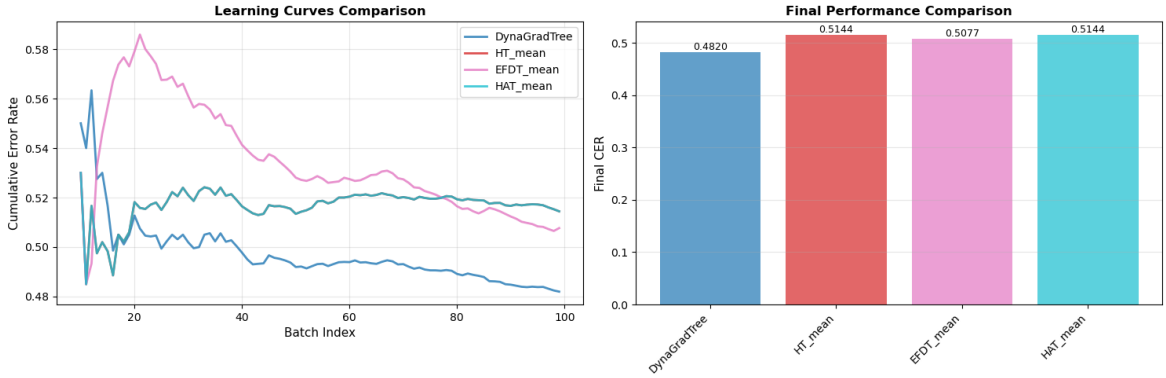


Figure E.3: Learning curve for Covertype dataset (54 features, 7 classes, 10,000 samples).

E.6 Letter Dataset The Letter dataset contains pixel-based features for recognizing handwritten capital letters. With 16 features derived from image statistics and 26 letter classes (A-Z), this dataset presents a challenging multi-class problem where discriminative features must separate visually similar letters.

E.7 Mushroom Dataset The Mushroom dataset contains categorical features describing physical characteristics of mushroom specimens, with the task of classifying mushrooms as edible or poisonous. With 22 categorical features such as cap shape, gill size, and odor, this dataset is notable for containing

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

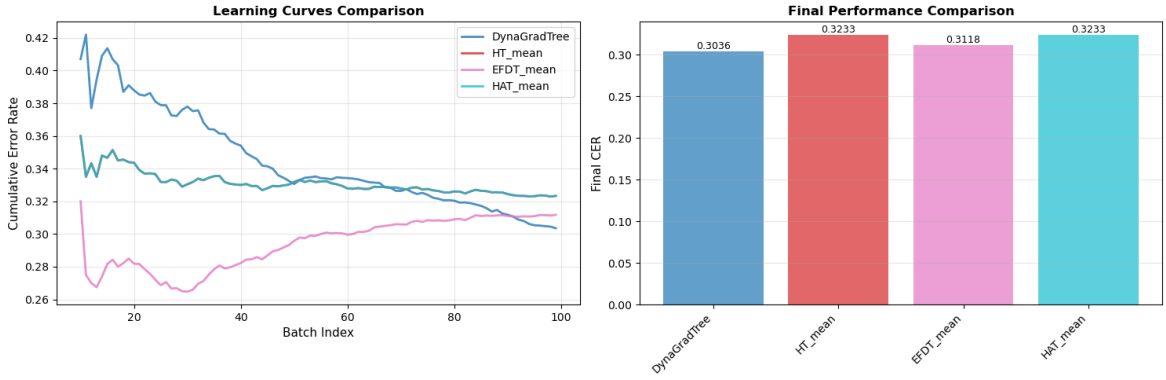


Figure E.4: Learning curve for Electricity dataset (8 features, 2 classes, 10,000 samples).

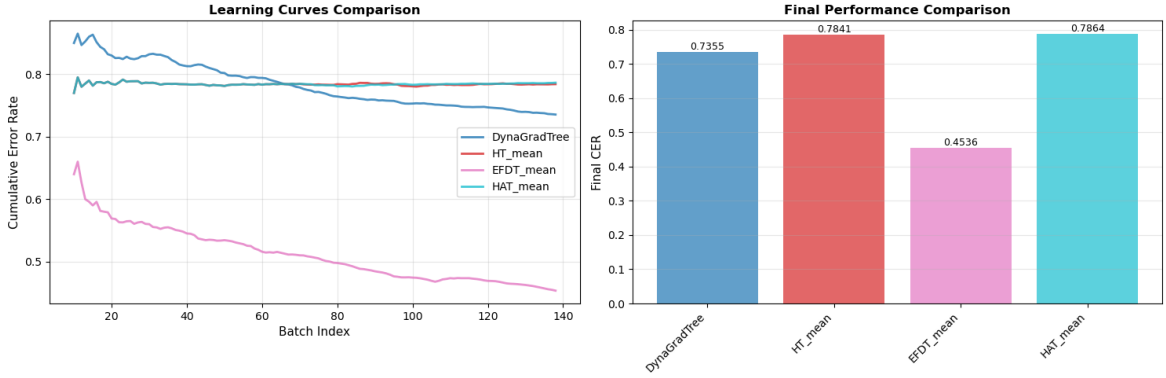


Figure E.5: Learning curve for Gas Sensor dataset (128 features, 6 classes, 10,000 samples).

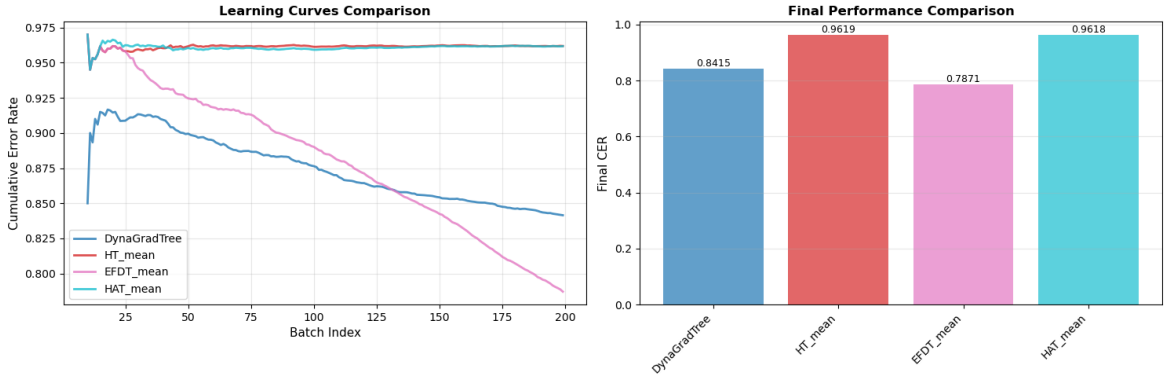


Figure E.6: Learning curve for Letter dataset (16 features, 26 classes, 10,000 samples).

rule-based structure that makes it amenable to decision tree methods.

E.8 Nomao Dataset The Nomao dataset originates from a website recommendation system with the task of binary classification. With 118 features representing various user interaction patterns and content attributes, this dataset presents a high-dimensional problem that tests scalability.

E.9 Pendigits Dataset The Pendigits dataset contains features extracted from pen-based handwritten digit recognition. With 16 features derived from pen trajectory data and 10 digit classes (0-9), this dataset

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

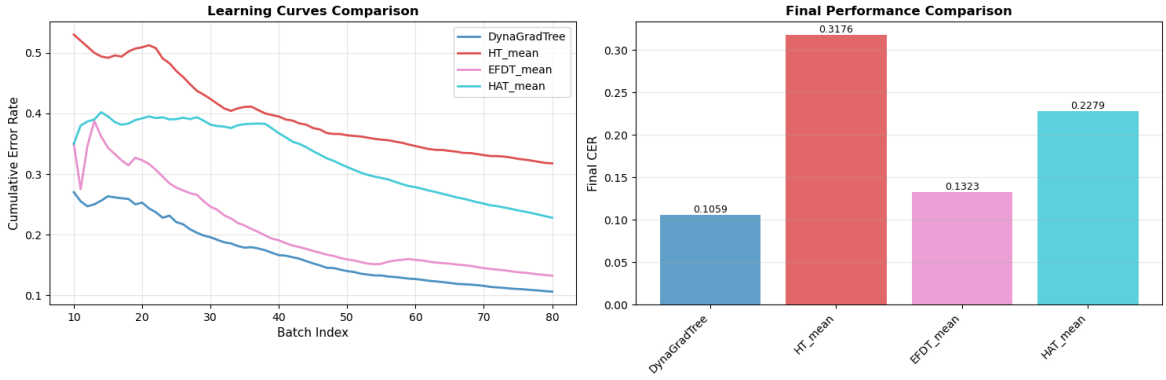


Figure E.7: Learning curve for Mushroom dataset (22 features, 2 classes, 8,124 samples).

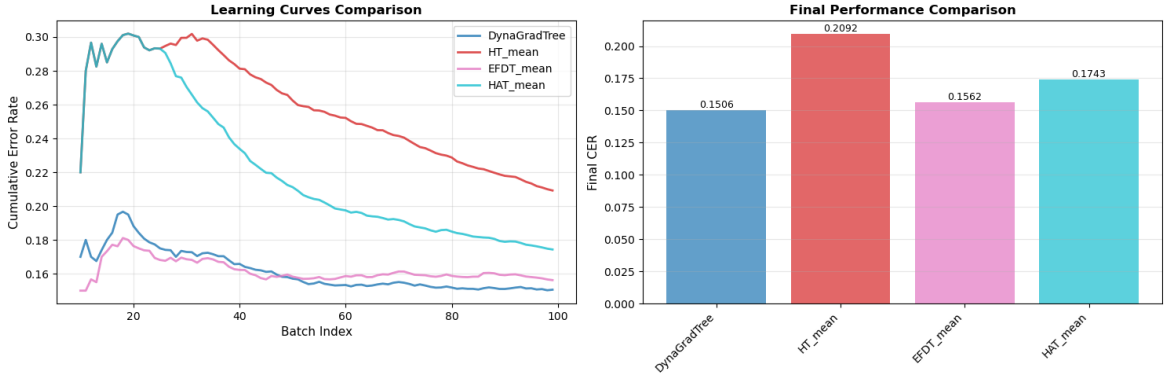


Figure E.8: Learning curve for Nomao dataset (118 features, 2 classes, 10,000 samples).

provides a moderate-dimensional multi-class classification task.

E.10 Phishing Dataset The Phishing dataset contains features extracted from website URLs and content for detecting phishing websites. With 9 features including URL characteristics, SSL certificate status, and page content properties, this dataset represents a security-critical binary classification task.

E.11 Poker Dataset The Poker dataset contains features describing poker hands, with the task of predicting the hand type. With 10 features (suit and rank for five cards) and 10 hand classes (from high card to royal flush), this dataset involves learning combinatorial rules from discrete features.

E.12 Satimage Dataset The Satimage dataset contains multi-spectral satellite image pixels with the task of classifying ground cover types. With 36 features representing spectral values across different wavelengths and 6 cover type classes, this dataset represents a moderate-dimensional multi-class problem.

E.13 Shuttle Dataset The Shuttle dataset contains features from NASA space shuttle missions for detecting different flight conditions. With 9 features representing sensor readings and 7 condition classes, this dataset is characterized by severe class imbalance.

E.14 Wall Robot Dataset The Wall Robot dataset contains ultrasonic sensor readings from a mobile robot navigating along walls, with the task of classifying the position of robot. With 24 features representing sensor measurements from different angles and 4 position classes, this dataset represents a robotics perception problem.

F Learning Curves: VFS Methods Comparison This section presents the complete learning curves for all 14 real-world datasets used in the comparison with VFS-specific methods, as reported in Section 5.2 and Table 5.2 of the main paper. The comparison includes four methods specifically designed to handle variable feature spaces: OLVF (Online Learning with Variable Features), OLSF (Online Learning with Shifting Features), ORF3V (Online Random Feature Forests

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

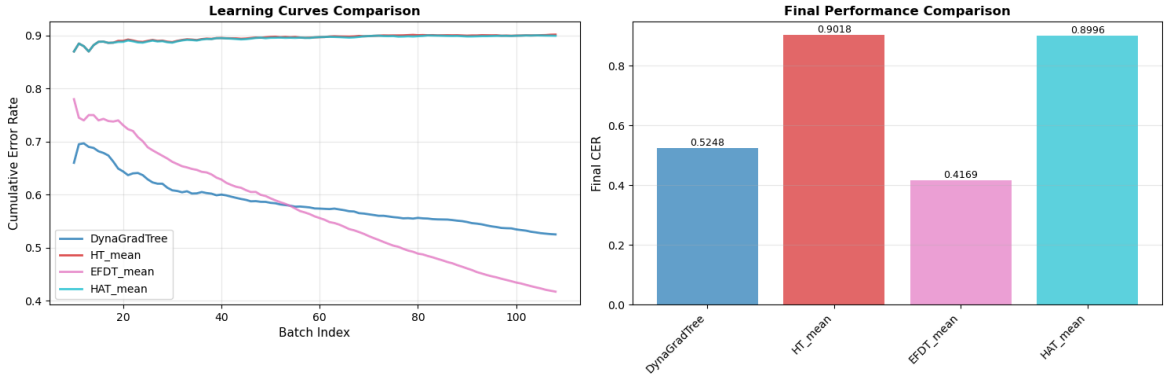


Figure E.9: Learning curve for Pendigits dataset (16 features, 10 classes, 10,000 samples).

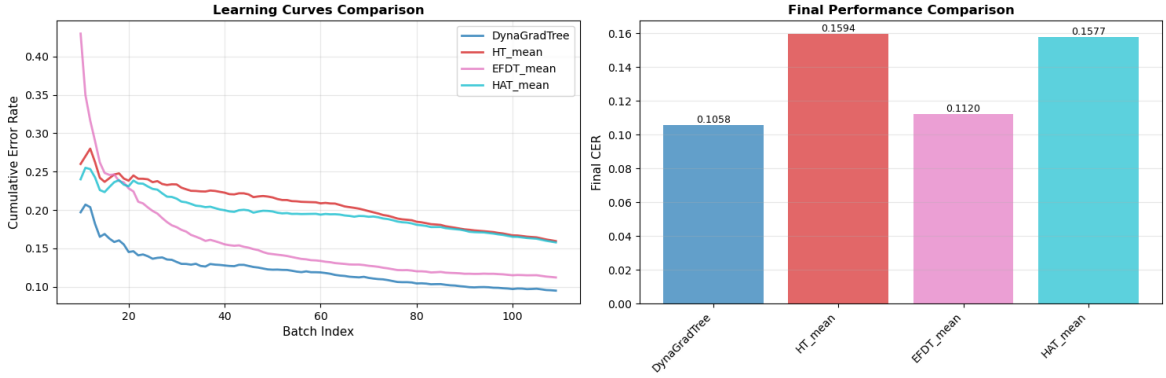


Figure E.10: Learning curve for Phishing dataset (9 features, 2 classes, 10,000 samples).

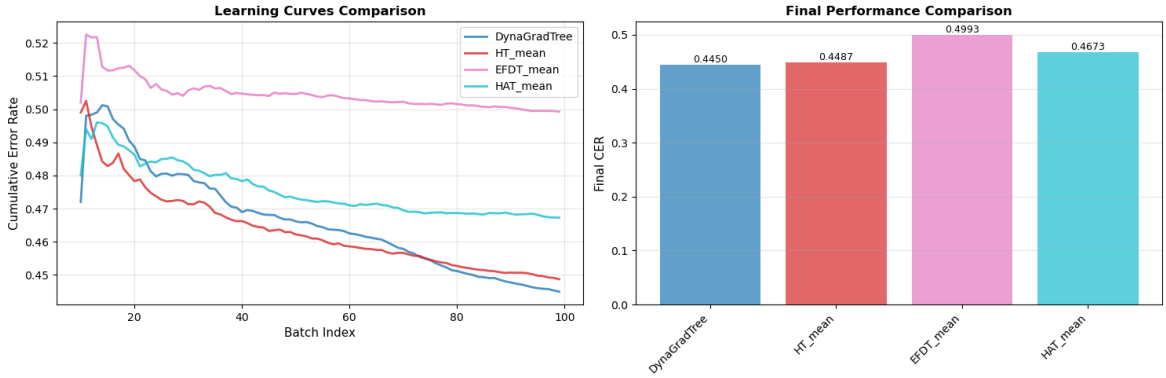


Figure E.11: Learning curve for Poker dataset (10 features, 10 classes, 10,000 samples).

for Feature space Variabilities), and OVFM (Online Variable Feature Model). The experimental setup uses the trapezoidal growth pattern where the proportion of available features gradually increases from 10% at the beginning to 100% at the final mini-batch, simulating scenarios where the feature space expands dynamically over time.

F.1 Adult Dataset Under the trapezoidal growth pattern, the Adult dataset begins with approximately 1-2 available features (10% of 14) and gradually expands to the full 14 features. This simulates scenarios where demographic data collection capabilities expand

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

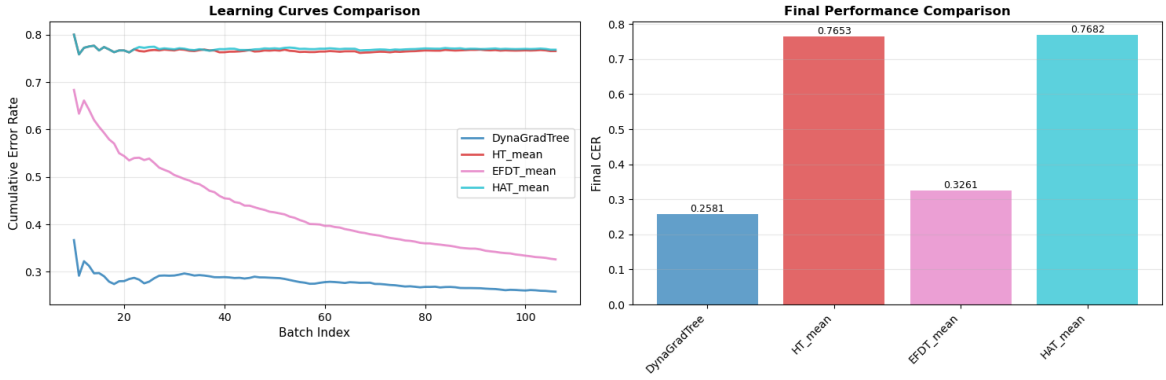


Figure E.12: Learning curve for Satimage dataset (36 features, 6 classes, 10,000 samples).

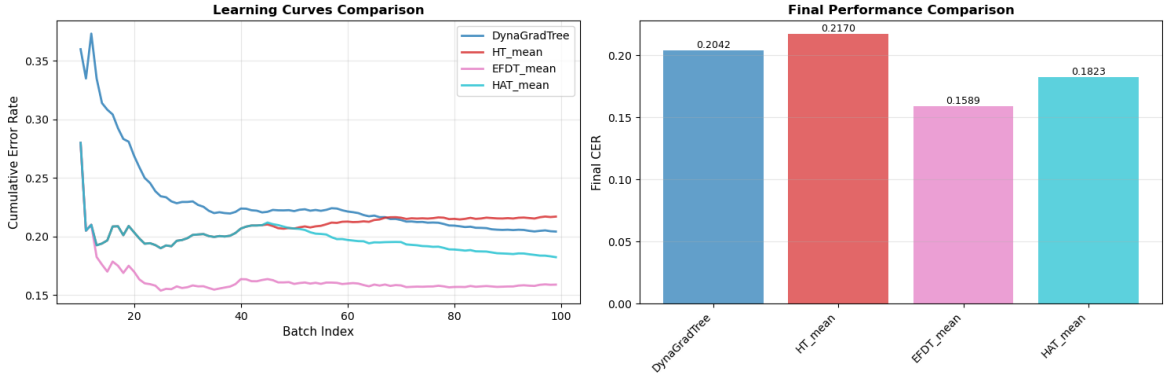


Figure E.13: Learning curve for Shuttle dataset (9 features, 7 classes, 10,000 samples).

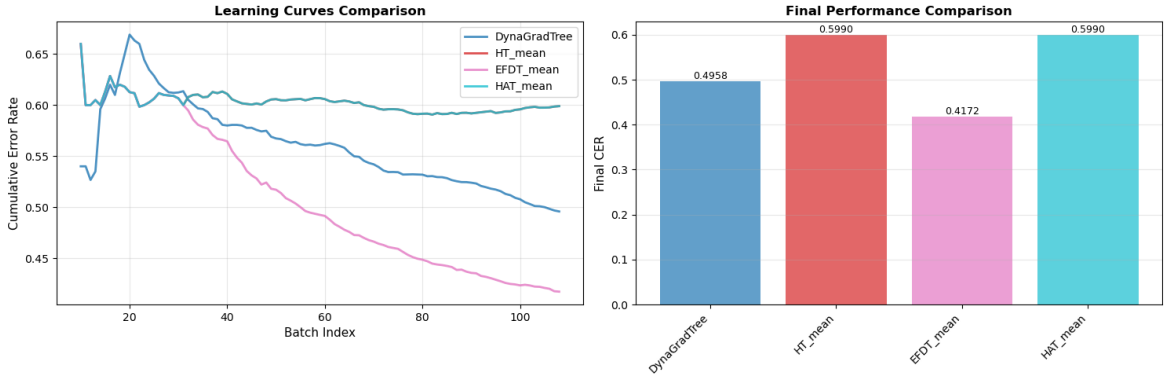


Figure E.14: Learning curve for Wall Robot dataset (24 features, 4 classes, 5,456 samples).

over time.

F.2 Airline Dataset With only 7 features total, the Airline dataset begins with a single feature and expands gradually. The low dimensionality provides a controlled setting for observing how methods adapt to feature space growth.

F.3 Covertype Dataset The Covertype dataset with 54 features provide substantial room for feature space expansion, starting from approximately 5 features and growing to the full set. This tests how methods handle moderate-to-high dimensional growth.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

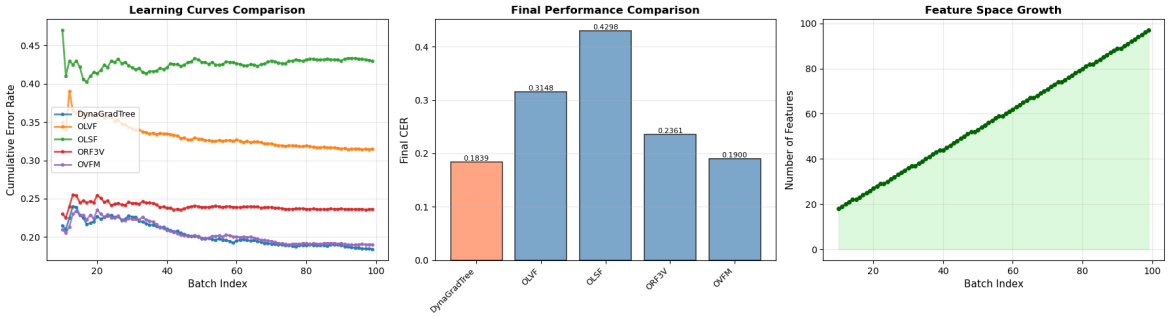


Figure F.1: Learning curve for Adult dataset in VFS comparison (14 features, 2 classes, 10,000 samples).

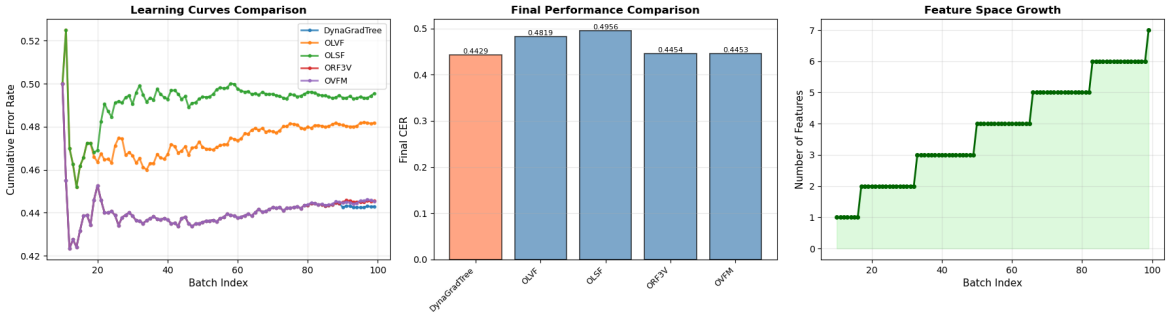


Figure F.2: Learning curve for Airline dataset in VFS comparison (7 features, 2 classes, 10,000 samples).

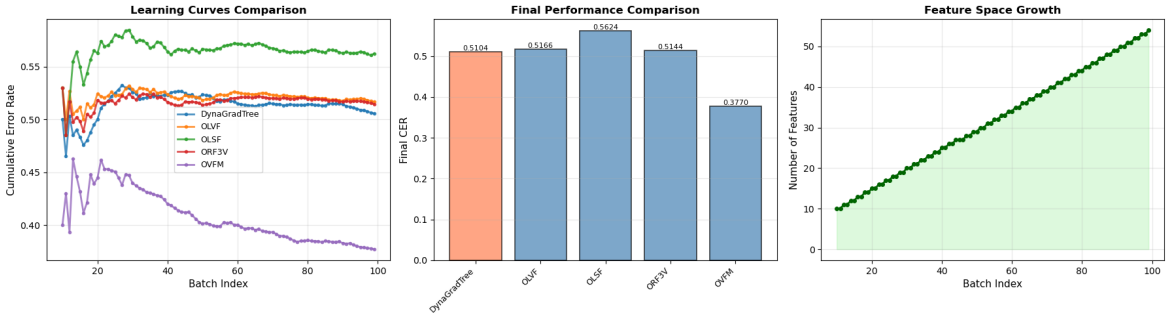


Figure F.3: Learning curve for Covertype dataset in VFS comparison (54 features, 7 classes, 10,000 samples).

F.4 Electricity Dataset The temporal dynamics of Electricity dataset combined with feature space growth creates a particularly challenging scenario where methods must simultaneously handle concept drift and expanding features.

F.5 Gas Sensor Dataset With 128 features, the Gas Sensor dataset represents an extreme test of VFS handling capabilities. Beginning with approximately 13 features and expanding to 128 requires methods to scale efficiently as dimensionality increases.

F.6 Letter Dataset The 26-class problem of Letter dataset, combined with growing features from 1-2 up to 16 tests whether methods can progressively refine class boundaries as more discriminative features become available.

F.7 Mushroom Dataset The categorical features and rule-based structure of Mushroom dataset provide an interesting test case for VFS methods. As additional categorical features become available, methods must integrate new decision criteria.

F.8 Nomao Dataset The 118 features of Nomao dataset provide extensive feature space growth, starting from approximately 12 features. The high dimensionality tests whether methods can maintain effi-

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

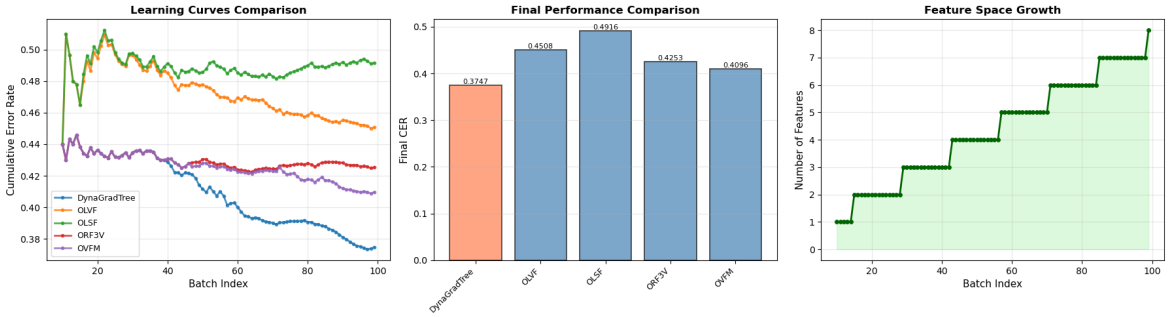


Figure F.4: Learning curve for Electricity dataset in VFS comparison (8 features, 2 classes, 10,000 samples).

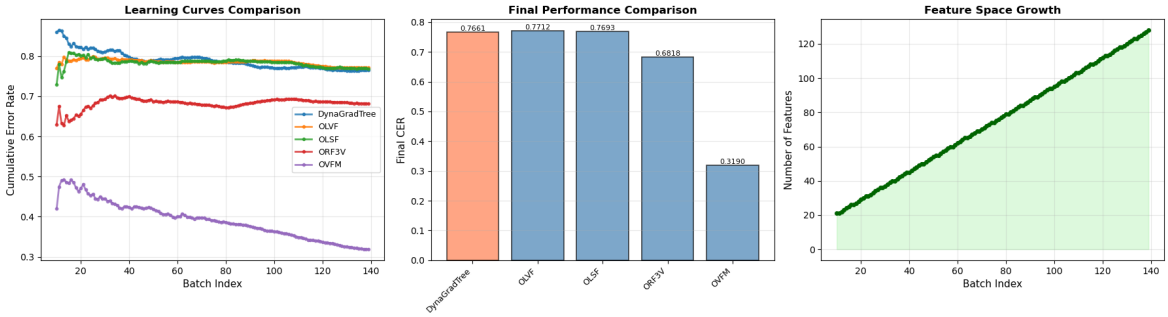


Figure F.5: Learning curve for Gas Sensor dataset in VFS comparison (128 features, 6 classes, 10,000 samples).

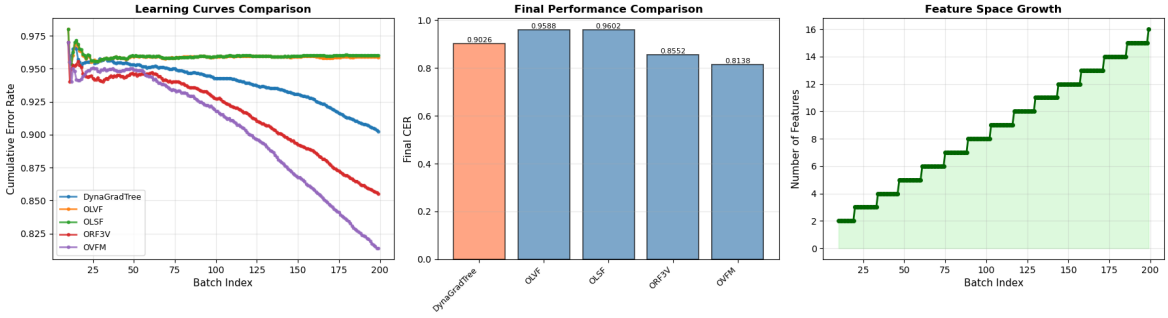


Figure F.6: Learning curve for Letter dataset in VFS comparison (16 features, 26 classes, 10,000 samples).

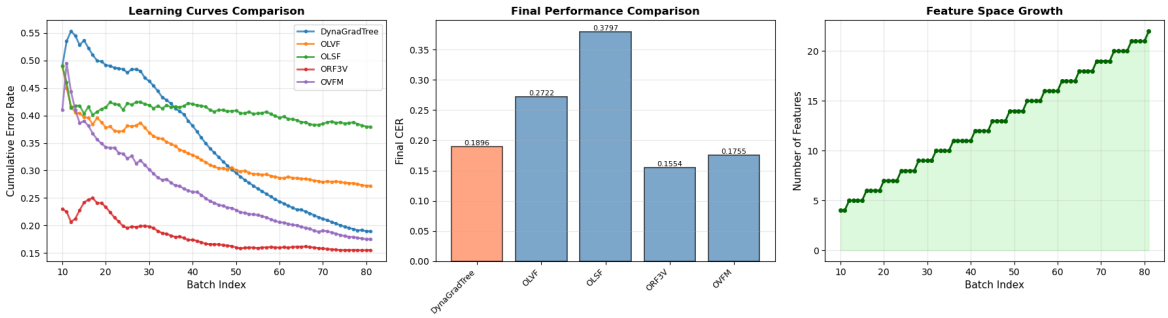


Figure F.7: Learning curve for Mushroom dataset in VFS comparison (22 features, 2 classes, 8,124 samples).

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

ciency as the feature space expands substantially.

F.9 Pendigits Dataset The Pendigits dataset with 16 features starts from 1-2 features and expands gradually. The digit recognition task tests how well methods can utilize additional discriminative features as they become available.

F.10 Phishing Dataset The security features of Phishing dataset expand from 1 to 9 features. The binary classification task tests how quickly methods can achieve reliable phishing detection as more URL and content features become available.

F.11 Poker Dataset The Poker dataset begins with information about 1-2 cards and gradually reveals all 5 cards. This simulates scenarios where poker hand information becomes available progressively during play.

F.12 Satimage Dataset The 36 spectral features of Satimage dataset expand from approximately 3-4 features at the start. This simulates scenarios where satellite imaging capabilities or spectral bands become available incrementally.

F.13 Shuttle Dataset The sensor readings of Shuttle dataset expand from 1 to 9 features, simulating scenarios where sensor arrays come online progressively or additional monitoring capabilities are added to flight systems.

F.14 Wall Robot Dataset The ultrasonic sensors of Wall Robot dataset expand from 2-3 active sensors to all 24 sensors. This simulates scenarios where robot sensor arrays are deployed incrementally or where sensor failures occur and recover.

G Ablation Study Learning Curves This section presents the complete set of learning curves for the ablation study examining the learned surrogate mechanism of DynaGradTree under different varying patterns and absence rates, as reported in Section 5.2 and Table 5.3 of the main paper. The ablation study compares three configurations: DynaGradTree with learned surrogates (full model), DynaGradTree with mean-filling instead of learned surrogates (ablation baseline), and DynaGradTree with zero-filling (naive baseline). Each experiment uses the synthetic dataset with 8,000 samples and 20 features organized into three correlated groups plus 14 independent features.

G.1 Feature Space Arbitrarily Varying The Random varying pattern removes features uniformly at random at each time step, with each feature having an independent probability of being absence. This pattern simulates noisy sensor readings, network packet loss, or transient system failures where absence data occurs unpredictably across all features.

G.2 Feature Absence w/ Recurring The feature absence with recurring pattern creates cyclical intervals where features alternately appear and disappear with regular frequency. This pattern simulates scenarios such as scheduled sensor maintenance windows or periodic system failures, where absence data occurs in predictable temporal patterns.

G.3 Feature Absence w/o Recurring The feature absence without recurring pattern maintains complete feature availability during initial training, then abruptly and permanently removes a subset of features at a specific time point. This pattern simulates catastrophic scenarios such as permanent sensor failures, system component degradation, or infrastructure damage.

G.4 Trapezoidal Absence Pattern The trapezoidal absence pattern gradually decreases the absence rate from high initial values to zero by the end of training. This pattern simulates scenarios where data collection systems mature and improve over time, such as sensor networks stabilizing after deployment or quality control processes being refined.

H Tree Structure Visualization Figure H.1 visualizes the complete tree structure learned by DynaGradTree under the Random-30% absence configuration after training on the synthetic dataset described in Section 5.2 of the main paper. This visualization provides empirical evidence for the observation stated in the main paper that the tree primarily relies on features from the three correlated groups (features 0-5) for split decisions.

H.1 Interpretation of Tree Structure The color-coded visualization reveals several important patterns that support the surrogate feature testing design. Primary features 0, 2, and 4 (shown in pink) appear frequently throughout the tree, particularly in the upper levels where they make the most influential splits. This is consistent with the higher importance weights assigned to the correlated groups in the synthetic dataset design. Surrogate features 1, 3, and 5 (shown in teal) also appear regularly across the tree structure, demonstrating that the model has learned to utilize these correlated features. The distribution of surrogate features across different levels suggests that they are not merely backup options but actively participate in the decision-making process. The 14 independent features (6-19) appear much less frequently in the tree structure, typically in deeper levels, validating the designed importance weighting where correlated groups were assigned higher importance than independent features.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

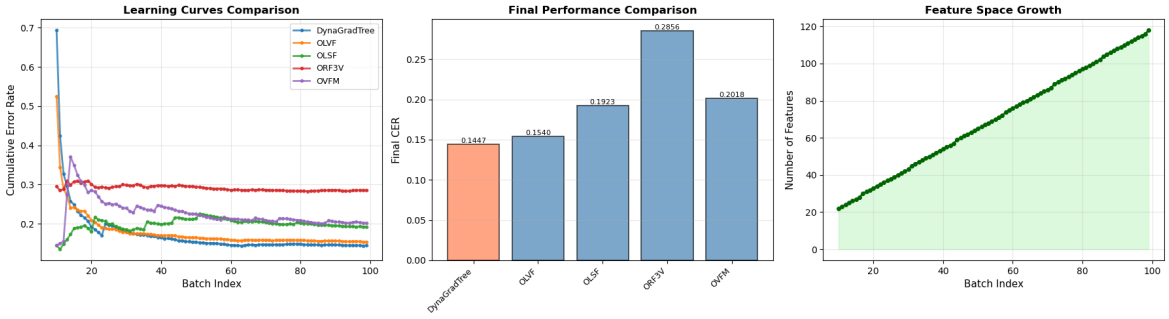


Figure F.8: Learning curve for Nomao dataset in VFS comparison (118 features, 2 classes, 10,000 samples).

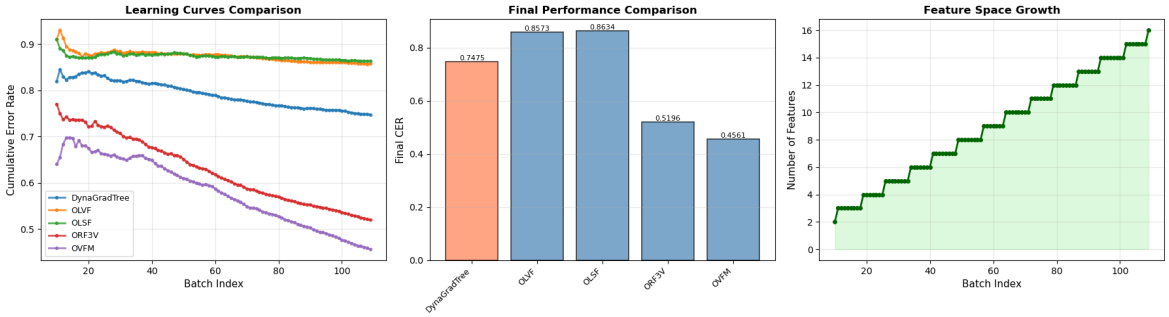


Figure F.9: Learning curve for Pendigits dataset in VFS comparison (16 features, 10 classes, 10,000 samples).

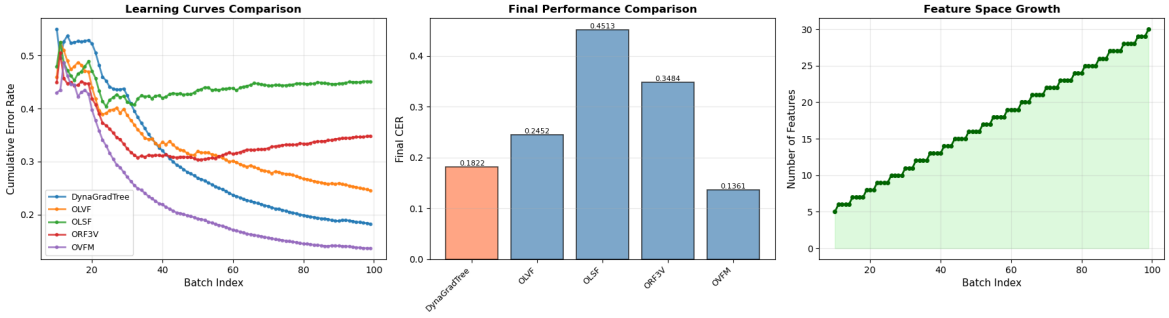


Figure F.10: Learning curve for Phishing dataset in VFS comparison (9 features, 2 classes, 10,000 samples).

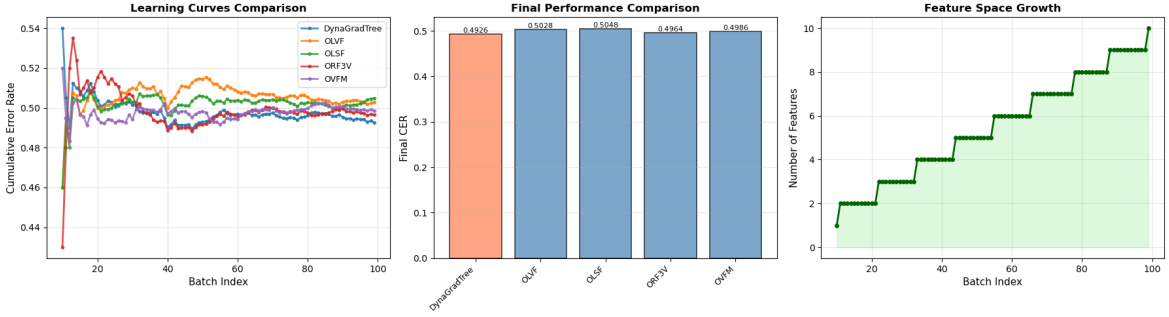


Figure F.11: Learning curve for Poker dataset in VFS comparison (10 features, 10 classes, 10,000 samples).

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

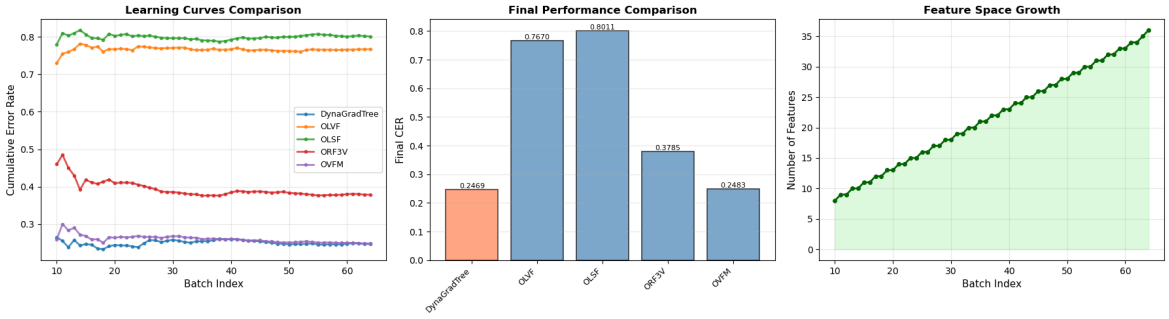


Figure F.12: Learning curve for Satimage dataset in VFS comparison (36 features, 6 classes, 10,000 samples).

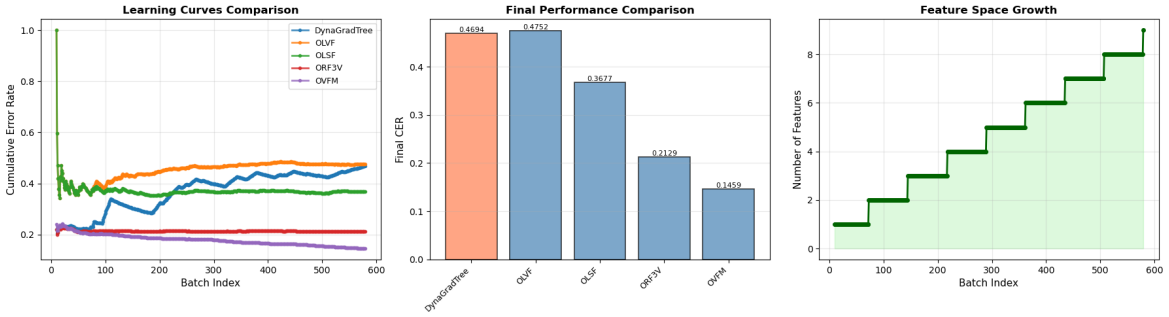


Figure F.13: Learning curve for Shuttle dataset in VFS comparison (9 features, 7 classes, 10,000 samples).

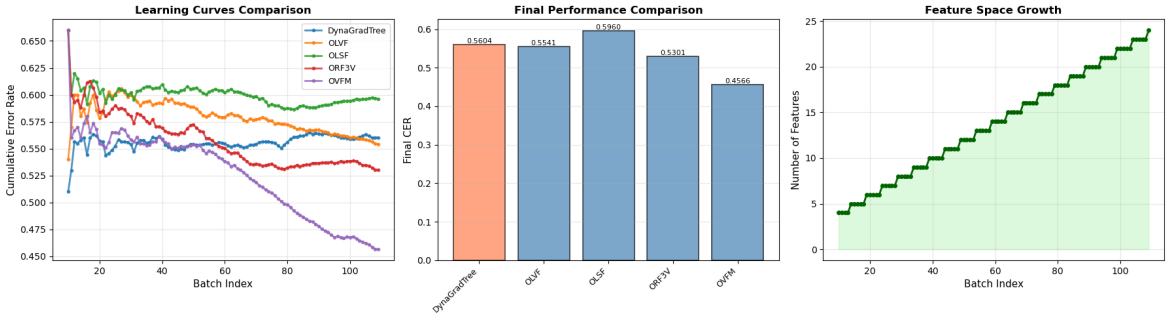


Figure F.14: Learning curve for Wall Robot dataset in VFS comparison (24 features, 4 classes, 5,456 samples).

H.2 Connection to Experimental Results This tree structure visualization provides theoretical foundation for interpreting the symmetric pattern in Table 5.4 of the main paper (RQ3). When primary features (0, 2, 4) are absent in Scenario 1, many nodes that normally would use these features must find alternatives. The learned surrogate mechanism allows the model to automatically switch to correlated features (1, 3, 5), resulting in the observed 1.25% point advantage over Mean-Filling. Conversely, when surrogate features (1, 3, 5) are absent in Scenario 2, the primary features (0, 2, 4) are still available at most nodes, so the surrogate mechanism does not need to activate as frequently, and performance becomes similar to Mean-Filling with only a 0.70% point difference. This asymmetry in the results directly reflects the asymmetry in feature usage patterns visible in the tree structure.

I Entmax Transformation for Soft Feature Selection The entmax transformation used in soft feature selection (Section 2.3) is a sparse, differentiable generalization of softmax introduced by Peters et al. (43). For a vector $\mathbf{z} \in \mathbb{R}^d$ representing feature importance scores and a parameter $\alpha \geq 1$, α-entmax is defined as the solution to the following optimization

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

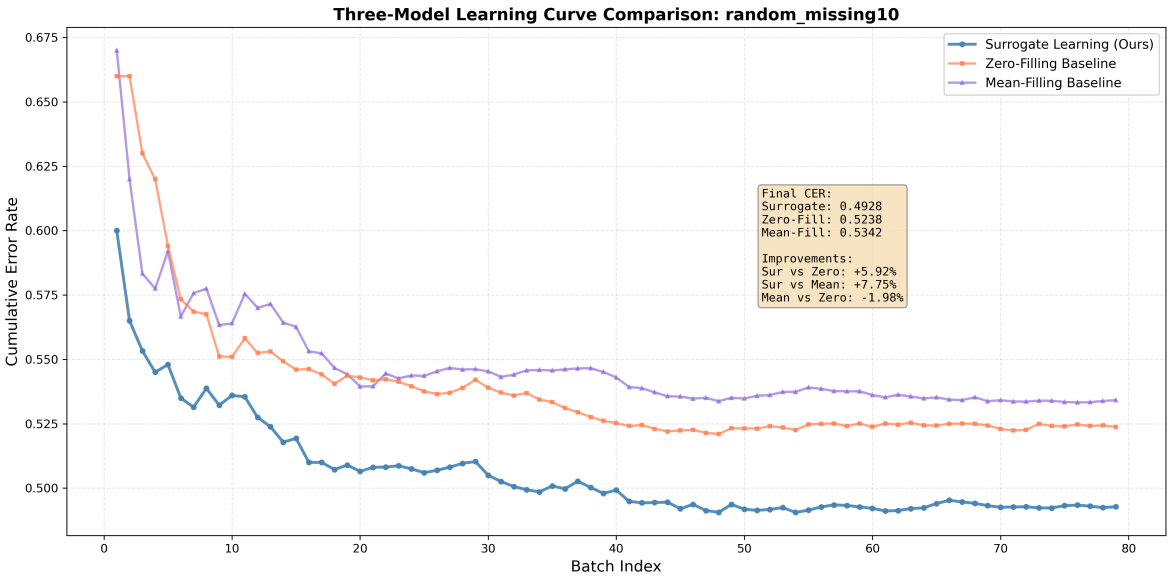


Figure G.1: Ablation study learning curves under Random absence pattern with 10% absence rate. Even at low absence rates, learned surrogates provide consistent advantages over mean-filling.

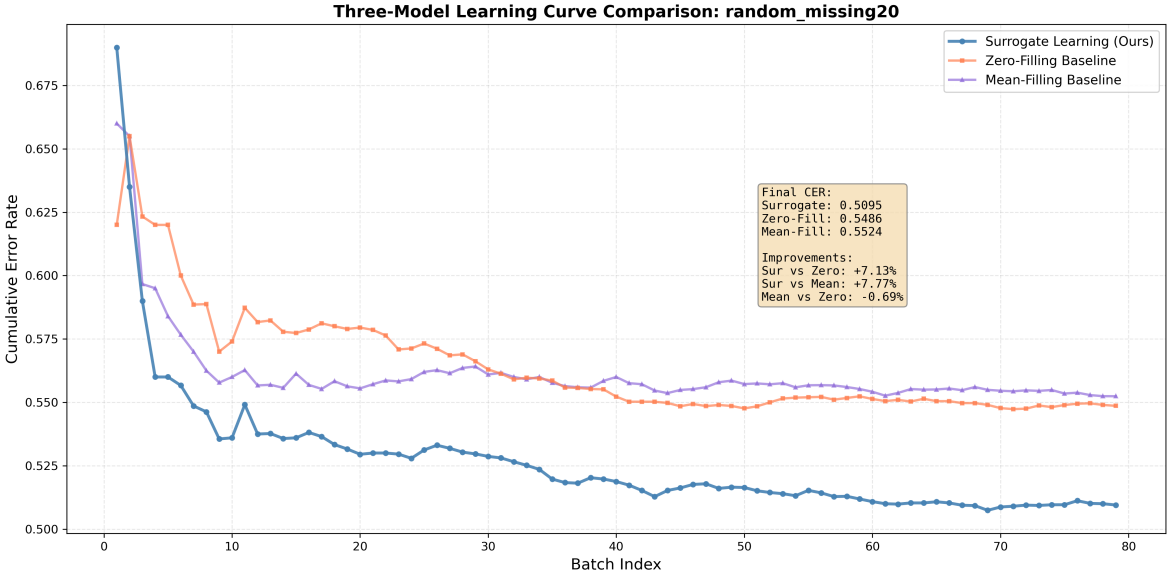


Figure G.2: Ablation study learning curves under Random absence pattern with 20% absence rate. The advantages of surrogate mechanism remain substantial as random absenceness increases.

problem over the probability simplex Δ^d:

$$\text{entmax}_\alpha(\mathbf{z}) = \arg\max_{\mathbf{p}\in\Delta^d} \left(\mathbf{p}^\top \mathbf{z} + H_\alpha(\mathbf{p})\right), \tag{I.1}$$

where H_α denotes the Tsallis α-entropy:

$$H_\alpha(\mathbf{p}) = \frac{1}{\alpha(\alpha-1)}\left(1 - \sum_{i=1}^{d} p_i^\alpha\right). \tag{I.2}$$

The solution to Eq. (I.1) admits the closed form:

$$[\text{entmax}_\alpha(\mathbf{z})]_i = [(\alpha-1)(z_i-\tau)]_+^{1/(\alpha-1)}, \tag{I.3}$$

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

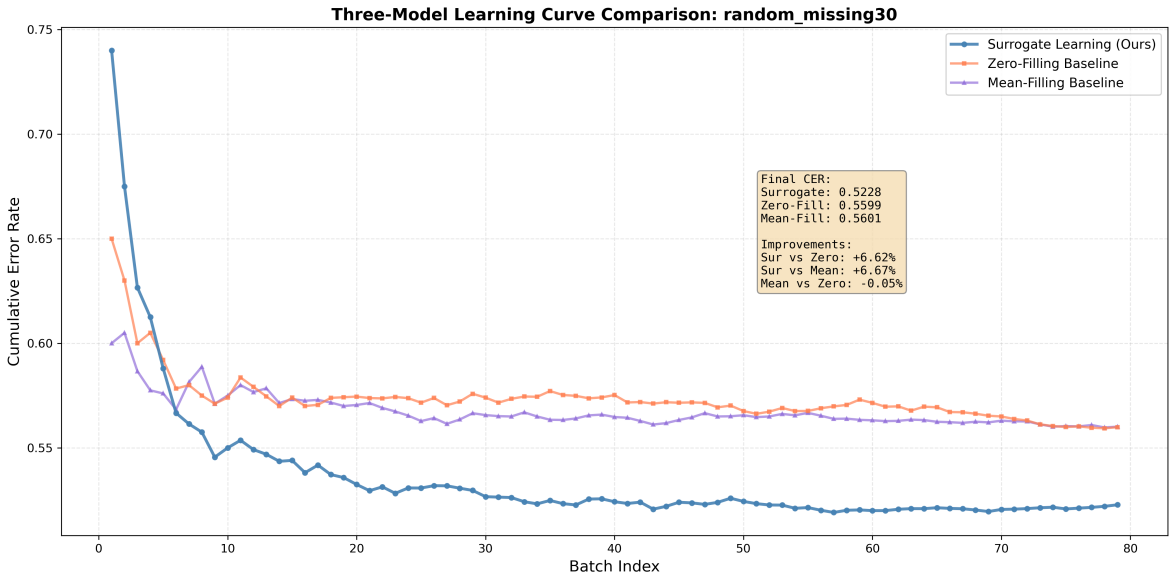


Figure G.3: Ablation study learning curves under Random absence pattern with 30% absence rate. At higher absence rates, the advantage of learned surrogates remains substantial.

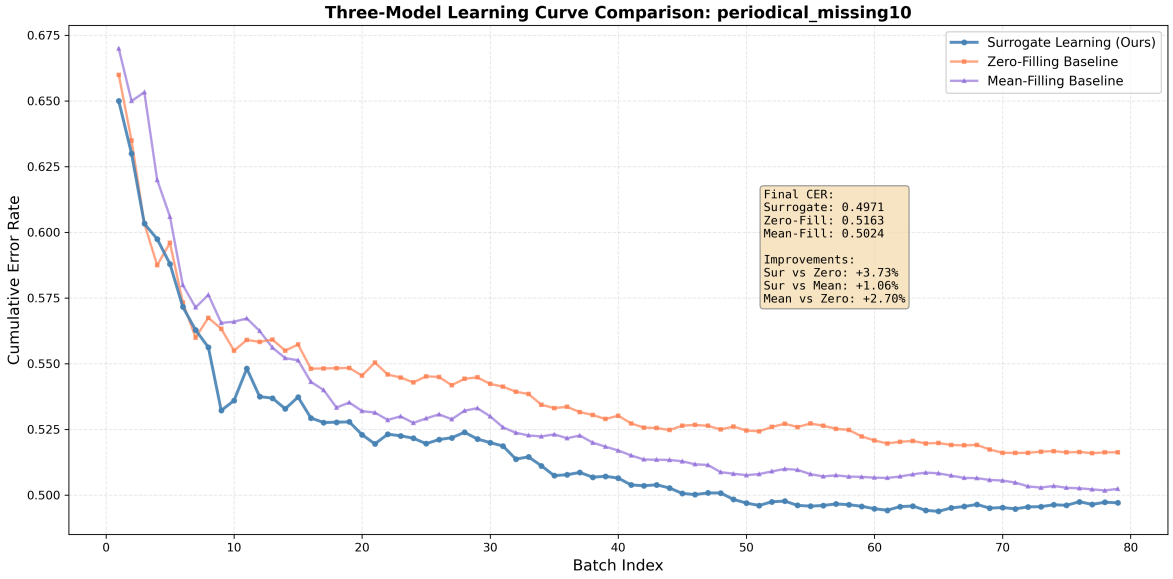


Figure G.4: Ablation study learning curves under feature absence with recurring pattern with 10% absence rate. The learning curves show mild fluctuations reflecting the cyclical nature of feature availability.

where $[\cdot]_+ = \max(0, \cdot)$ and $\tau \in \mathbb{R}$ is the unique scalar satisfying $\sum_{i=1}^{d} [\text{entmax}_\alpha(\mathbf{z})]_i = 1$.

Two standard special cases are obtained at the boundary values: $\alpha \to 1$ recovers the ordinary softmax, while $\alpha = 2$ yields sparsemax (39), which projects $\mathbf{z}$ onto Δ^d and can assign exact-zero probability to low-scoring entries. Following GradTree (40), DynaGradTree uses $\alpha = 1.5$, a value that interpolates between the two extremes and has been found to produce distributions that are sparse enough to approximate discrete feature selection while remaining sufficiently smooth for stable gradient-based optimization.

In the context of DynaGradTree, the input vector $\mathbf{z}$ at each node v corresponds to the feature importance vector $\mathbf{w}_v$ (Eq. (3) in the main paper). The entmax transformation converts these importance scores into a

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

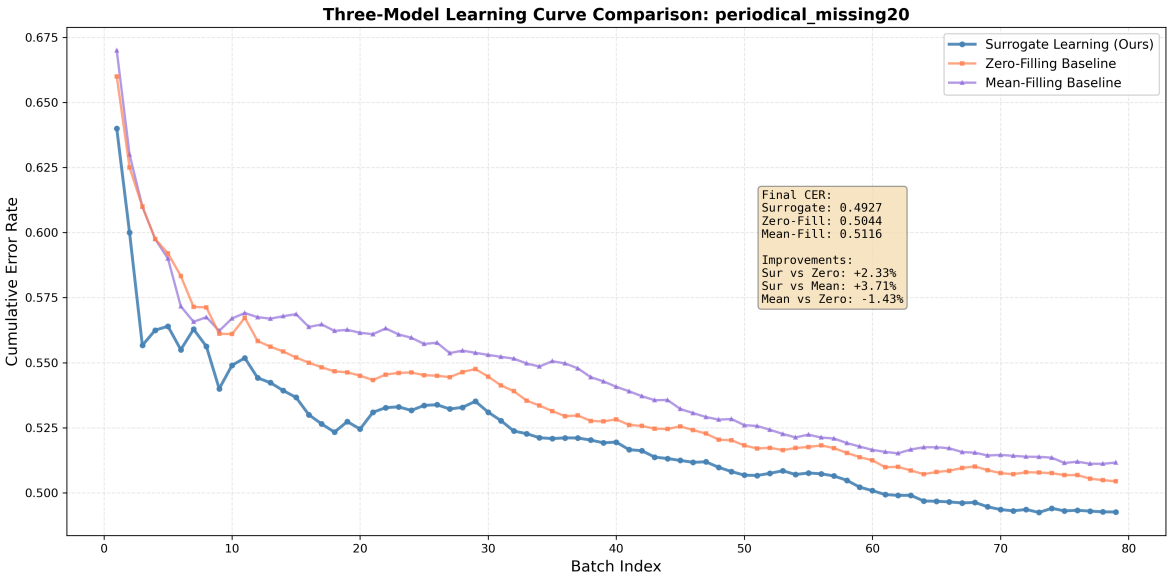


Figure G.5: Ablation study learning curves under feature absence with recurring pattern with 20% absence rate. DynaGradTree maintains more stable performance across the periodic availability cycles.

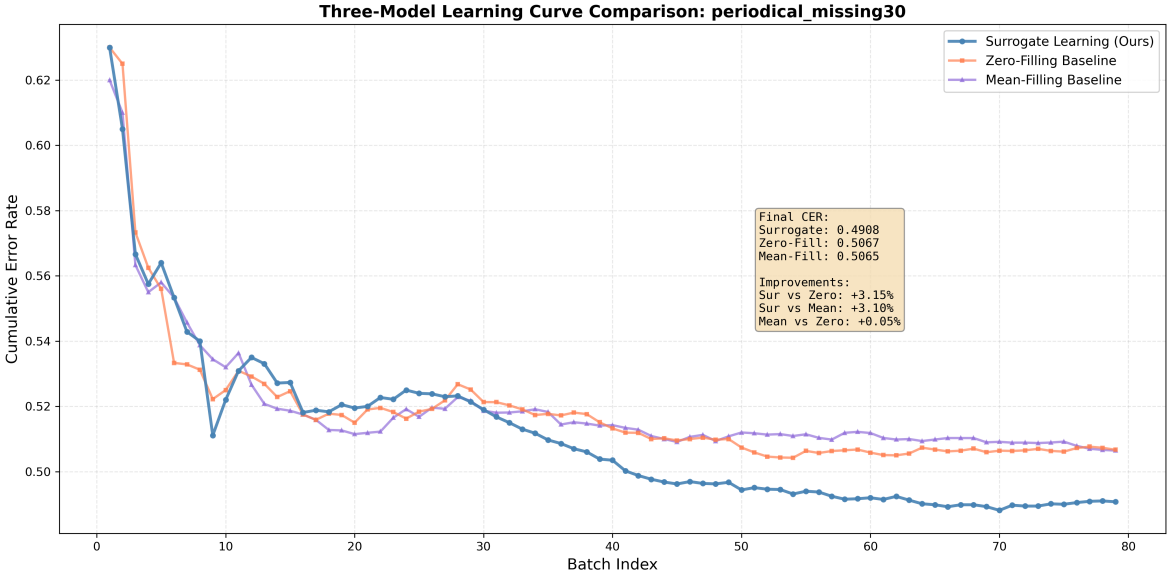


Figure G.6: Ablation study learning curves under feature absence with recurring pattern with 30% absence rate. The learned surrogate mechanism demonstrates robust adaptation to the cyclical absence pattern.

probability distribution $\mathbf{p}_v = \text{entmax}_{1.5}(\mathbf{w}_v)$ over features, where each $p_{v,i}$ represents the probability of selecting feature i at node v. The sparsity-inducing property of entmax with $\alpha = 1.5$ means that typically only a small subset of features receive non-zero probabilities, naturally implementing a soft form of feature selection.

The gradient of entmax with respect to its input is well-defined and can be computed efficiently. For $\mathbf{p} = \text{entmax}_\alpha(\mathbf{z})$, let $S = \sum_{k:p_k>0} p_k^{2-\alpha}$ denote the normalizing constant. The Jacobian $\mathbf{J} = \frac{\partial \mathbf{p}}{\partial \mathbf{z}}$ is then given by:

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

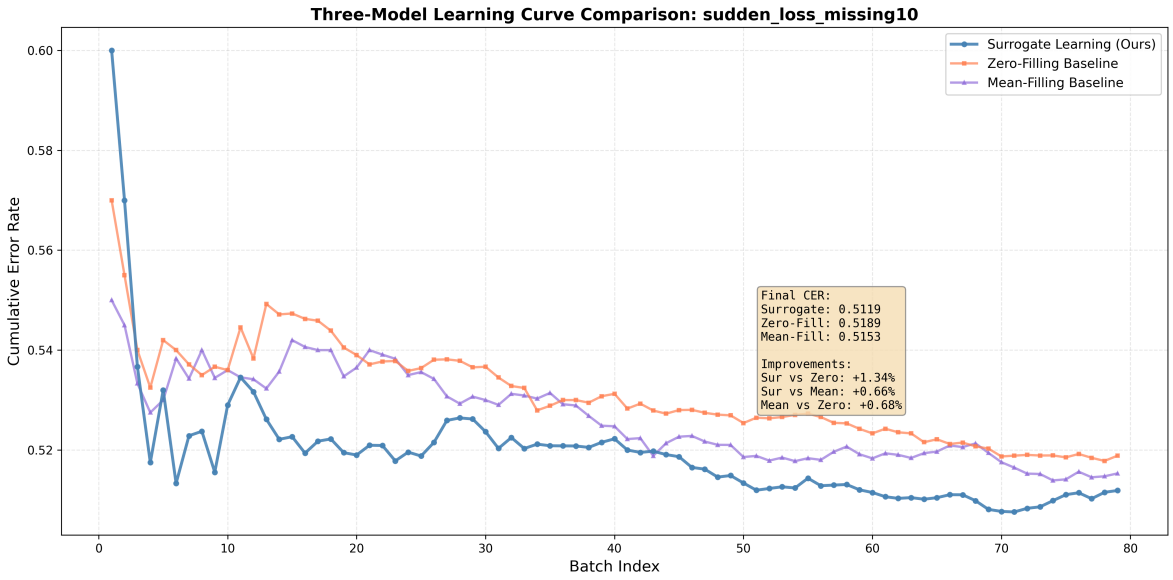


Figure G.7: Ablation study learning curves under feature absence without recurring pattern with 10% absence rate. The curves show performance degradation at the point of sudden feature loss, followed by recovery as the model adapts.

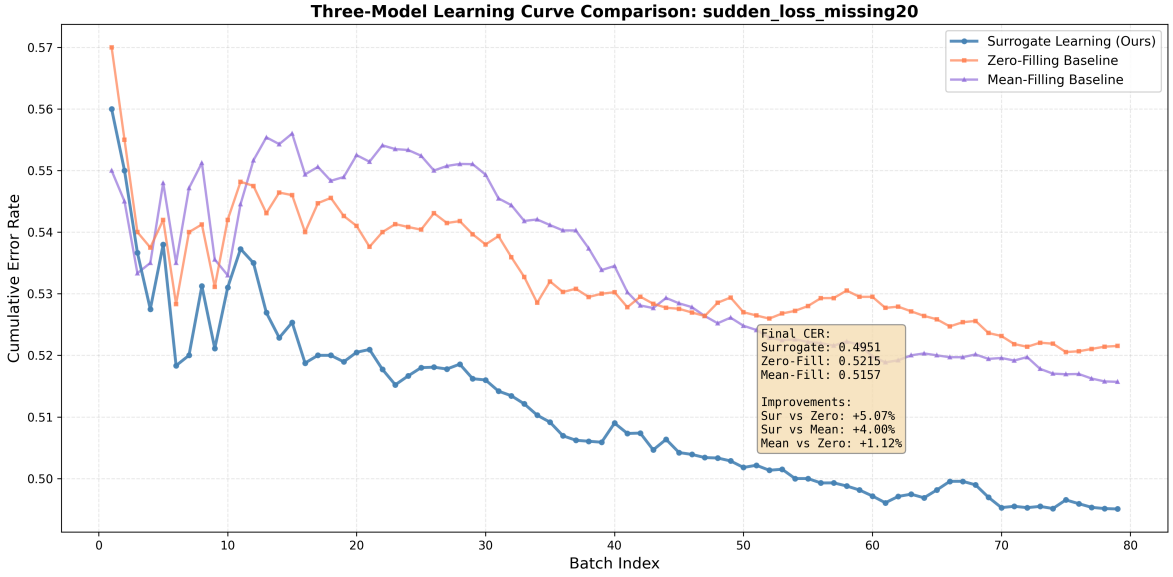


Figure G.8: Ablation study learning curves under feature absence without recurring pattern with 20% absence rate. DynaGradTree demonstrates faster recovery from the sudden feature loss compared to mean-filling.

(I.4)

$$J_{ij} = \frac{\partial p_i}{\partial z_j} = \begin{cases} p_i^{2-\alpha}/S & \text{if } i = j \text{ and } p_i > 0 \\ -p_i^{2-\alpha} p_j^{2-\alpha}/S^2 & \text{if } i \neq j \text{ and } p_i, p_j > 0 \\ 0 & \text{otherwise} \end{cases}$$

This sparse Jacobian structure makes backpropagation through entmax computationally efficient, as gradients need only be computed for features that received non-zero probability in the forward pass. During training, DynaGradTree leverages this property to update feature importance weights $\mathbf{w}_v$ via standard gradient descent, allowing the model to learn which features are most discriminative at each node while maintaining dif-

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

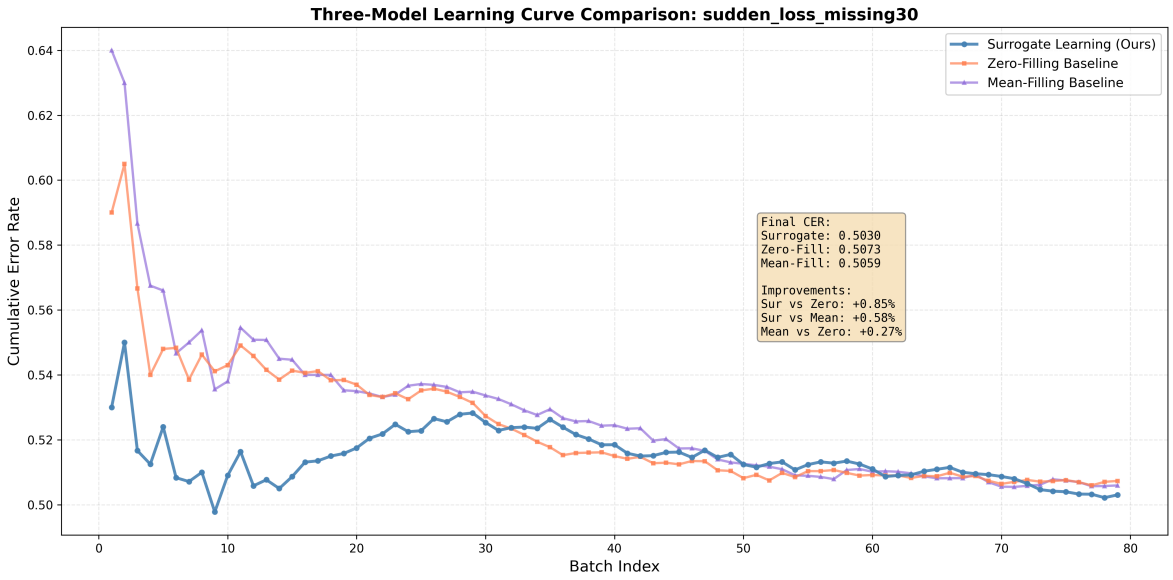


Figure G.9: Ablation study learning curves under feature absence without recurring pattern with 30% absence rate. The learned surrogate mechanism enables more effective early adaptation to the permanent loss of features.

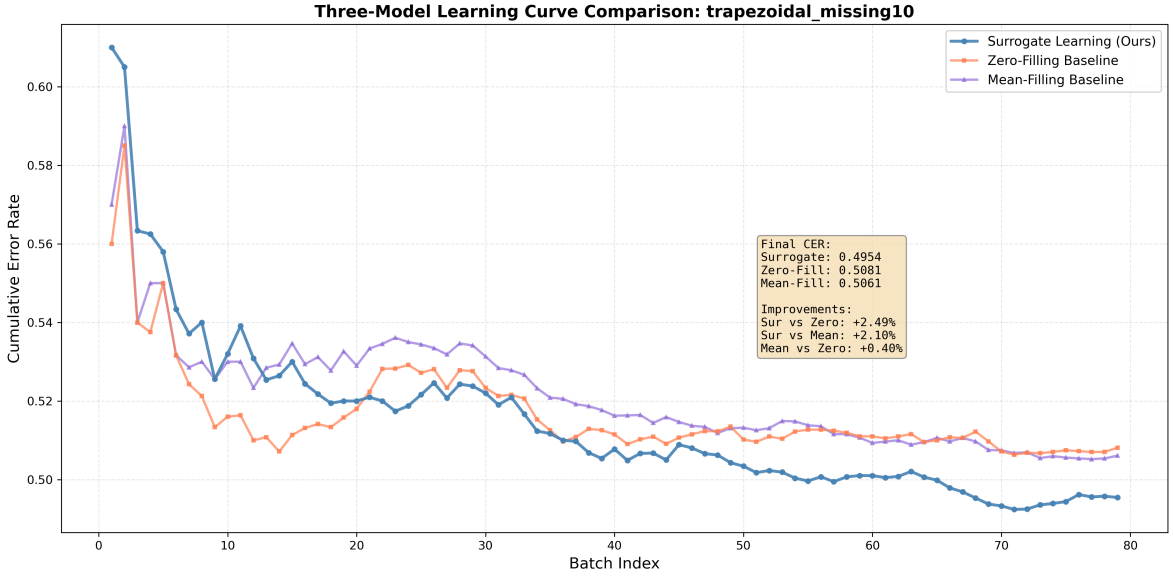


Figure G.10: Ablation study learning curves under trapezoidal absence pattern starting with 10% absence rate and gradually improving to 0%. All methods show improvement as data quality increases.

ferentiability throughout the optimization process.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

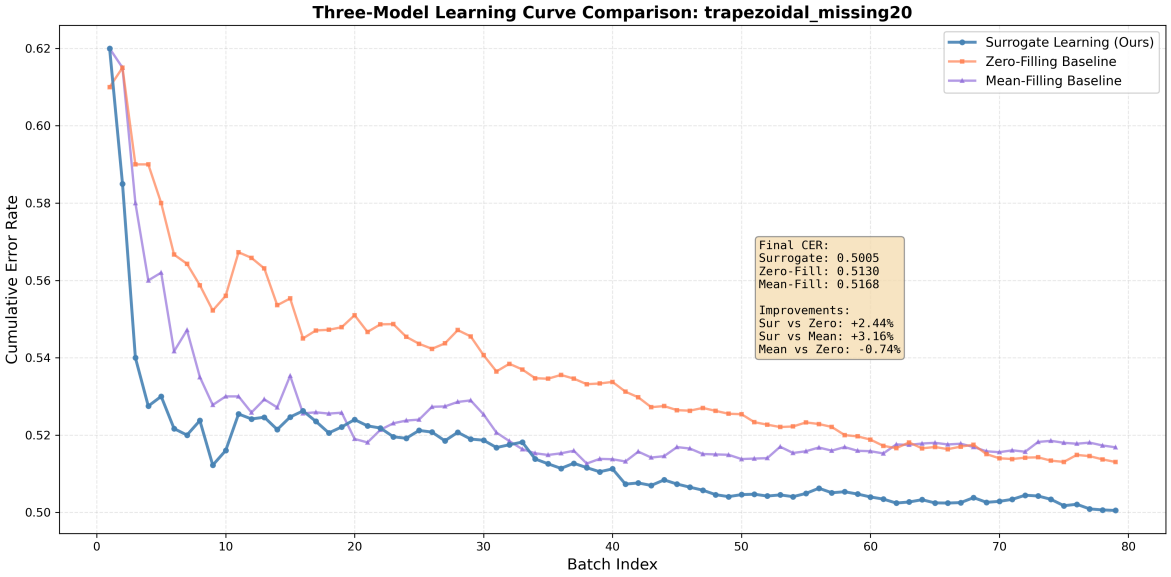


Figure G.11: Ablation study learning curves under trapezoidal absence pattern starting with 20% absence rate and gradually improving to 0%. DynaGradTree maintains consistent advantages throughout the improving data quality trajectory.

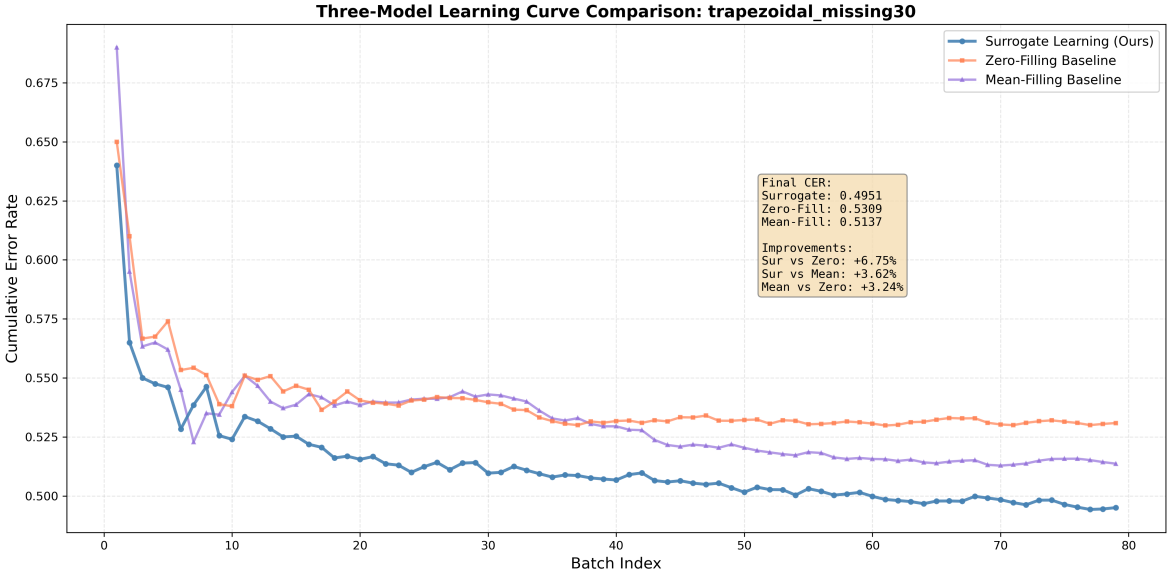


Figure G.12: Ablation study learning curves under trapezoidal absence pattern starting with 30% absence rate and gradually improving to 0%. The performance gap to mean-filling narrows as absence rate approaches zero and the need for surrogates diminishes.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited

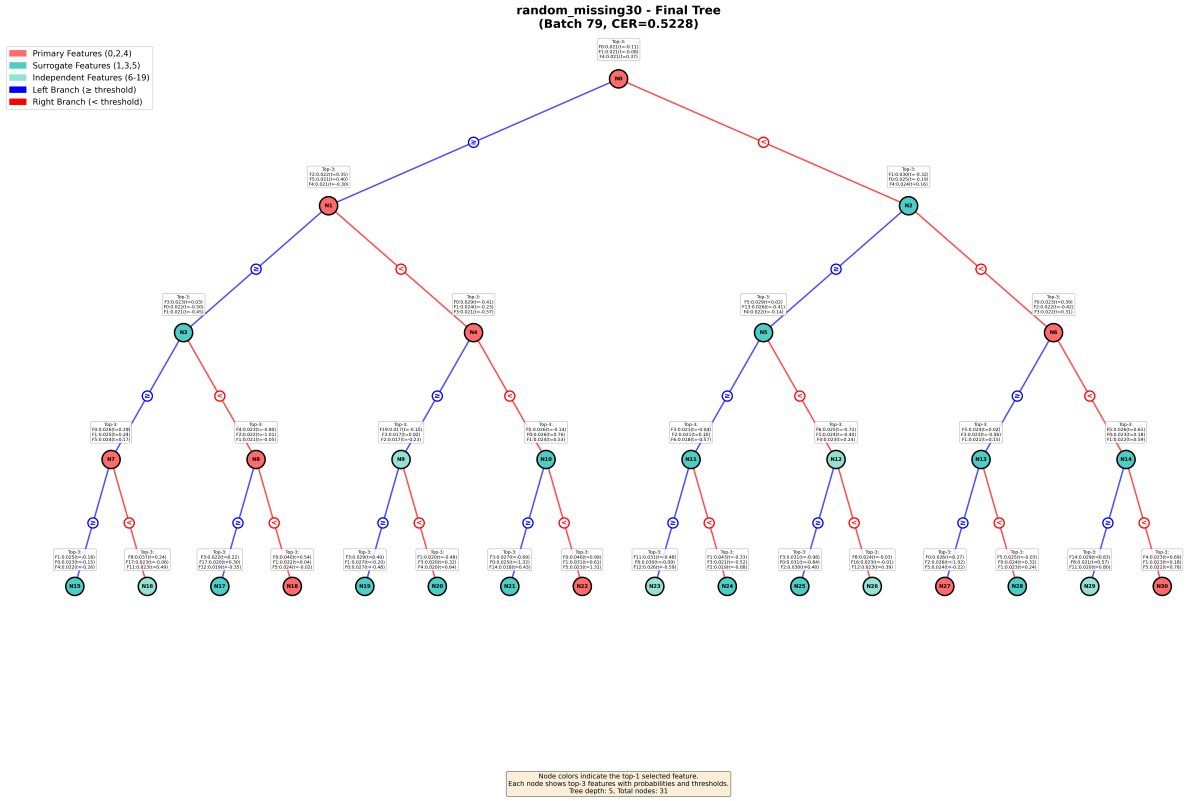


Figure H.1: **Tree structure learned from the Random-30% absence configuration.** The tree was trained on a synthetic dataset with 8,000 samples and 20 features. Node colors indicate the selected feature at each split: pink nodes use primary features (0, 2, 4), teal nodes use surrogate features (1, 3, 5), and other colors indicate independent features (6-19). The visualization demonstrates that the vast majority of internal nodes utilize features from the three correlated feature groups (features 0-5), with only a small number of nodes selecting from the 14 independent features. This empirical observation motivated the design of the Primary Features absence and Surrogate Features absence test scenarios presented in Table5.4 of the main paper.

Copyright © 2026 by SIAM
Unauthorized reproduction of this article is prohibited